

基于优化搜索的烟雾弹投放问题分析

摘要

烟雾弹是一种有效的干扰武器，无人机可以通过投放烟雾弹在目标特定区域形成遮蔽，干扰敌方导弹对我方实施的轰炸。本文针对烟雾干扰弹经历的四个阶段：平动、平抛、爆炸、消弭，建立了运动模型和遮挡模型。随后自主设计了一种搜索算法，实现了对单个烟雾弹遮挡问题的全局寻优，并评估该种算法的性能。又利用其他优化搜索算法，对多导弹、多无人机、多抛射问题进行了分析。

针对问题一：构建无人机、烟雾弹与导弹飞行模型。首先在题目坐标系的前提下，通过物理模型，计算出在投弹爆炸的时候导弹与烟雾弹所处的具体位置，对导弹和烟雾弹的轨迹进行分析求解。再通过视线-球体遮挡判定的一个数学函数作为判断条件，对所处情况进行判定。在有效爆炸时长内，利用二分法求解时间子区间并进行并集计算，从而得到有效遮蔽总时长。

针对问题二：先分析一种简化后的二维情形，通过简化模型出现的问题，给出修正该理论的一种方法。根据对一般方程建模的分析学研究，提出在三维空间中爆炸点对应遮挡时间的特点。借助几何分析，又给出最前时间原则，剖析了该问题下烟雾弹爆炸时间的要求。接下来，一方面，结合上述引理，给出一个创新的算法，并给出自创的该算法用到的数学工具，分析优越性并得出结果；另一方面，我们研究了变延迟策略优化算法对应的结果，对这两种算法进行了比较分析。

针对问题三：系统分析了其算法选择及原因，给出目标函数。接下来，结合实际需要，分别给出线性约束和非线性约束的约束条件。出于算法应用的需要，给出超参数的设定及其原因。接下来，利用上述算法求解问题，并将该算法与问题二当中的算法进行深入比较。

针对问题四：在问题一对于无人机, 烟雾弹, 坐标系与导弹飞行模型的基础上, 处理三架无人机投弹进行分析: 三架无人机分别投弹, 对目标导弹的遮蔽应理解为三枚烟雾弹对导弹遮蔽时间的并集 (存在同一时间大于等于 2 枚烟雾弹都在遮蔽导弹的可能性), 关于最后的结果, 应当小于三枚烟雾弹对于导弹的遮蔽时间之和 (存在时间重叠), 对此我们将此问题的三架无人机的位置, 速度, 方向, 投弹时间等都独立地放置在问题二 (单个无人机单枚烟雾弹场景) 下进行仿真, 以获取单架无人机的遮蔽时间. 构建三架无人机同时遮蔽的场景, 与单架无人机遮蔽相比较, 以分析结果合理性.

针对问题五：系统分析了该问题的复杂性和主要矛盾，采用遗传算法进行分析。接下来给出该算法的实现细节，特别是染色体的编码方法。最后，得到最终结果并对最终结果进行验证，论证了结果的有效性。

关键字： 优化搜索 遮挡 贝叶斯优化

目录

一、问题背景与重述	3
1.1 问题背景	3
1.2 问题要求	3
二、问题分析	4
2.1 问题一的分析	4
2.2 问题二的分析	4
2.3 问题三的分析	5
2.4 问题四的分析	5
2.5 问题五的分析	5
三、模型准备	6
3.1 模型假设	6
3.2 符号说明	6
四、模型的建立与求解	6
4.1 问题一	6
4.1.1 问题一模型的建立	6
4.1.2 问题一模型的求解	7
4.2 问题二	10
4.2.1 问题二模型的建立	10
4.2.2 问题二模型的求解	12
4.3 问题三	15
4.4 问题四	17
4.5 问题五	18
参考文献	19
附录 A 4.2 中两个运动方程组的解	20
附录 B 问题一—matlab 源代码	21
附录 C 问题一—python 源代码	22

附录 D	问题二–python 源代码 1	26
附录 E	问题二–python 源代码 2	31
附录 F	问题三–matlab 源代码	43
附录 G	问题三–python 源代码	47
附录 H	问题四–python 源代码	54
附录 I	问题五–python 源代码	61
附录 J	python 环境配置的说明	70

一、问题背景与重述

1.1 问题背景

我国军事力量不断向智能化方向发展，2025 年的九三阅兵更是显示了这一趋势。现今，目标遮蔽已经成为军事战争中的一种重要手段，无人机投放烟雾弹便是实现智能化的一种方案。保证成本低、费用比高两大优势的同时，可以通过投放烟雾对目标进行遮蔽，并有效干扰敌方导弹。在此条件下，调整无人机的飞行速度方向、投弹时间和爆炸时间，可以进一步增长有效遮蔽时间。若无人机或导弹数目不唯一，无人机投弹的相应参数会相互影响，可能产生其他可能，这就要求我们具体分析每种情况，实现算法设计。

1.2 问题要求

现有 5 架无人机 FY1,FY2,FY3,FY4,FY5 和三枚导弹 M1,M2,M3，原点处为一个假目标，真目标为一个半径为 7m，高为 10m 的一个圆柱，下底面在 (0,200,0)，导弹的飞行方向指向原点，速度为 300m/s，无人机在收到任务后能以 70~140m/s 的速度等高度飞行匀速直线飞行，烟雾弹脱离无人机后在重力作用下运动，一段时间后爆炸，然后云团以 3m/s 速度匀速下落，云团中心 10m 内可有效遮挡，时长 20s，两枚烟雾弹之间的投放时间至少 1s，

我们需要解决以下问题：

1. 现在只考虑无人机 FY1 和导弹 M1 并且只发生一枚烟雾弹，FY1 现在朝假目标以 120m/s 的速度飞行，在接收到任务 1.5s 后投弹，然后间隔 3.6s 后起爆，现在建立出合适的数学模型，求出从开始飞行到爆炸结束过程中，烟雾能有效遮蔽真目标的时长。
2. 仍然只考虑无人机 FY1 和导弹 M1，但是此时无人机的飞行方向，飞行速度，投弹时间，爆弹时间均未知，如何调整这四项，使得烟雾遮挡真目标的有效时间最长。

3. 在只考虑无人机 FY1 和导弹 M1 的基础上，从 1 枚烟雾弹变成了 3 枚烟雾弹，同样，无人机的飞行方向，飞行速度，投弹时间，爆弹时间均未知，如何做出合理地策略规划，从而使得烟雾遮挡真目标的有效总时长最长，注意 FY1 投放两枚烟雾弹的时间间隔至少 1s。
4. 现考虑 3 架无人机 FY1、FY2、FY3，每架投一枚烟雾弹，实施对导弹 M1 的干扰。无人机的飞行方向，飞行速度，投弹时间，爆弹时间均未知，注意此时不需要考虑投弹时间间隔限制，但是要考虑三架无人机的投放时间与投放位置。
5. 一般化，现考虑 5 架无人机，每架无人机至多投放 3 枚烟雾弹，实施对于三枚导弹 M1、M2、M3 的干扰，无人机的飞行方向，飞行速度，投弹时间，爆弹时间均未知，如何做出合理地策略规划，从而使得烟雾遮挡真目标的有效总时长最长，注意 3 个导弹都需遮挡。

二、问题分析

2.1 问题一的分析

对于问题一，在已知了无人机和导弹的具体飞行情况后，我们可以先确定在烟雾弹爆炸之前，无人机、烟雾弹、导弹的运动情况和到达位置，在抛单时间，爆弹时间和无人机飞行方向速度已知的前提下，借助物理的抛体运动知识和数学分析求得状况。

在爆弹之后，便要开始计算时间，首先得计算导弹和云团的运动轨迹，于是可以得到导弹与真目标之间的视野直线方程，而云团随时间变化的中心也已知，则可以求出视线与球心的距离，并与有效遮挡半径进行比较进行判定，列出约束条件后再通过二分法求解时间子区间并取并集，从而得出有效遮挡总时长。

2.2 问题二的分析

问题二需优化无人机飞行方向、速度及烟雾弹投放 / 爆弹时间，以最大化单枚烟雾弹对导弹视线的有效遮蔽时长。核心矛盾在于“模型精度”与“优化效率”的平衡：初始可将问题简化为二维场景（导弹、无人机与假目标共面），此时烟雾弹最优爆弹位置对应“烟雾球面与视线直线相切”，但该简化忽略了真目标与假目标的纵坐标偏差，导致模型存在系统误差。

从优化特性看，遮蔽时长随无人机参数（速度、方向）和烟雾弹时序参数（投放、爆弹时间）的变化呈“多峰值函数”特征，即三维空间内存在有限个极值点，且极值点附近函数震荡平缓。若直接采用全局搜索，会因维度高、计算量大导致效率低下；若仅依赖局部迭代，又易陷入局部最优。因此需通过“降维转换”（如将空间位置映射为圆柱面参数 λ 、 θ ）简化优化对象，并结合我们给出的“最前时间原则”（优先选择早爆弹

方案)确定全局优化方向,在保证模型精度的同时,规避无效搜索区域,平衡多样性与收敛效率。最后可以给出一个原创的优化算法,进行迭代并最终达到全局最优。

2.3 问题三的分析

问题三扩展为单无人机投放三枚烟雾弹,核心挑战源于多弹时序耦合与约束复杂性,此即三枚烟雾弹的投放或爆弹时间相互影响,且总遮蔽时长为各弹遮蔽区间的并集而非简单相加,这导致目标函数呈现混沌性,单个参数的微小变化可能引发总时长的非线性波动,问题二的最前时间原则不再适用。

从优化方法看,需处理“多变量+多约束”的非线性优化问题:变量含无人机速度、方向及6个时序参数(3个投放时间、3个引信时延),约束含线性边界(如速度、时间范围)与非线性耦合(如投放间隔)。因此可以选择序列二次规划算法,进行全局寻优。

2.4 问题四的分析

对于问题四,其与问题二高度相似:均为单个无人机携带单枚烟雾弹进行遮蔽,但唯一不同点在于:问题四的三枚烟雾弹可能在同一时间对导弹进行遮蔽,此处时间只能计算一次,而不能够将所有无人机的遮蔽时间简单相加,由于仍然是多峰值函数以及多参数优化问题,拟采用贝叶斯优化,(使用贝叶斯优化算法的优势在于可以对目标函数的不确定性进行建模,并利用已有的观测数据来更新模型的预测。可以在高维空间中有效地搜索最优解,并在较少的评估次数下找到较好的解决方案,尤其适合高维度的参数优化)使得三架无人机的各项参数都使用贝叶斯算法优化,优化目标为导弹M1被遮蔽的总时长。

其中,物理模型的构建与问题2高度相似,遮蔽条件在下文”模型的建立与求解”中有详细数学推导与规定,具体优化算法和问题二采用的算法存在差异(但问题2本身使用了遗传进化算法,贝叶斯优化算法以及前文提及的我们自主研发的数学分析算法,得出的结果差异均小于0.1s,可以充分认为,算法的正确性是能够保证的)

2.5 问题五的分析

问题五为多无人机、多烟雾弹与多导弹的综合遮蔽优化,核心在于解决“多主体参数耦合”与“多目标遮蔽协同”的双重复杂问题。需协调多架无人机的飞行参数与多枚烟雾弹的投放时序,同时满足多枚导弹的遮蔽需求,且各参数间存在物理约束与逻辑关联,优化空间维度显著提升,约束关系更趋交织。

其核心难点在于,一是变量维度高,涵盖无人机飞行参数与烟雾弹时序参数,且需关联多枚导弹的遮蔽需求,变量间耦合性强;二是约束复杂,除基础边界约束外,还存在无人机任务分配、烟雾弹投放间隔、多导弹遮蔽时序衔接等隐性约束,约束冲突风险

增加；三是目标函数非凸性突出，总遮蔽时长为多导弹遮蔽区间并集的总和，随参数变化呈多峰、震荡特征，局部最优解数量多。而贝叶斯优化在多约束、强耦合场景下，模型不确定性易累积，难以精准定位可行域；梯度类方法受限于目标函数可微性，无法应对遮蔽区间并集带来的不连续问题。而遗传算法通过基因编码将无人机-烟雾弹参数映射为染色体，可直接处理高维变量与复杂约束，结合锦标赛选择策略平衡种群多样性与优解保留，再通过交叉、变异机制实现全局搜索与局部微调，能有效适配该问题的特性。

三、模型准备

3.1 模型假设

- **忽略空气阻力：**烟雾弹抛出后做忽略空气阻力的平抛运动，不受地球重力场以外任何物质的影响。
- **多主体协同假设：**假设无人机和烟雾弹足够小，多个无人机和烟雾弹在同一空间内协作时不会发生冲突。
- **导弹优先级假设：**假设各个导弹优先级一致，不存在需要重点防御和遮挡某个导弹的情况。
- **视线遮挡判定假设：**我们假设视线被遮挡，当且仅当观察者与被观察者之间的连线穿过烟雾。

3.2 符号说明

所使用的符号及说明如表 1 所示。

四、模型的建立与求解

4.1 问题一

4.1.1 问题一模型的建立

首先我们先对导弹、无人机与烟幕弹的运动建模。关于该问题，为方便读者理解，我们采用了 python 仿真的方式，给出了关于导弹与烟雾弹运动表征的可视化表示。

首先，我们给出了烟幕弹投放的全过程。如图 1 所示，烟幕弹的投放分为四个阶段：平动、平抛、下沉、消弭。这四个状态依次进行，其中间的过渡事件分别为抛射、爆炸、爆炸后 20s。

然而，仅由烟雾弹的运动观察问题是不全面的，我们需要观察烟雾弹与导弹的交互过程，如图 2（a）、2（b）所示。

根据上述对运动状态的分析，我们进行几何建模如下。

表 1 物理量符号说明

符号	意义	单位
t_1	烟雾弹抛出之前所用的时间	s
t_2	烟雾弹抛出到爆炸经过的时间	s
t_3	烟雾弹爆炸之前所用的时间	s
t_4	烟雾弹恰好停止遮挡单个导弹之前所用的时间	s
(x_m, y_m, z_m)	导弹的坐标	(m,m,m)
(x_0, y_0, z_0)	圆柱面上点的坐标	(m,m,m)
λ	圆柱面上点向主轴的投影与原点的距离	m
θ	圆柱面上点的旋转角度	rad
v_{di}	无人机 FYi 的飞行速度	m/s
θ_{di}	无人机 FYi 的飞行角度	m/s

4.1.2 问题一模型的求解

(1) 起爆时导弹与云团位置的确定

由已知导弹的初始位置为 P_{M_0} 和导弹的运行方向（指向原点），可以得到导弹的运动方向为：

$$\vec{u_M} = -\frac{P_{M_0}}{|P_{M_0}|} \quad (1)$$

在导弹速度大小为 v_m 的情况下，可以得到在烟雾弹起爆时，即经历了 $(t_1 + t_2)$ 的时间后导弹所在的位置为：

$$M(t_1 + t_2) = P_{M_0} + v_m (t_1 + t_2) \vec{u_M} \quad (2)$$

而对于云团的位置，则要分成两个部分进行计算，首先是无人机的初始位置为 P_{F_0} 跟随着无人机进行运动，投弹时已经运动了 t_1 时间，速度大小为 v_F ，方向为 $\widehat{u_F}$ ，则投弹时烟雾弹的位置为：

$$F(t_1) = P_{F_0} + \widehat{u_F} v_F t_1 \quad (3)$$

投弹后，烟雾弹进行平抛运动。在重力的作用下，烟雾弹在 xOy 平面作匀速直线运

烟雾弹投放全流程示意图

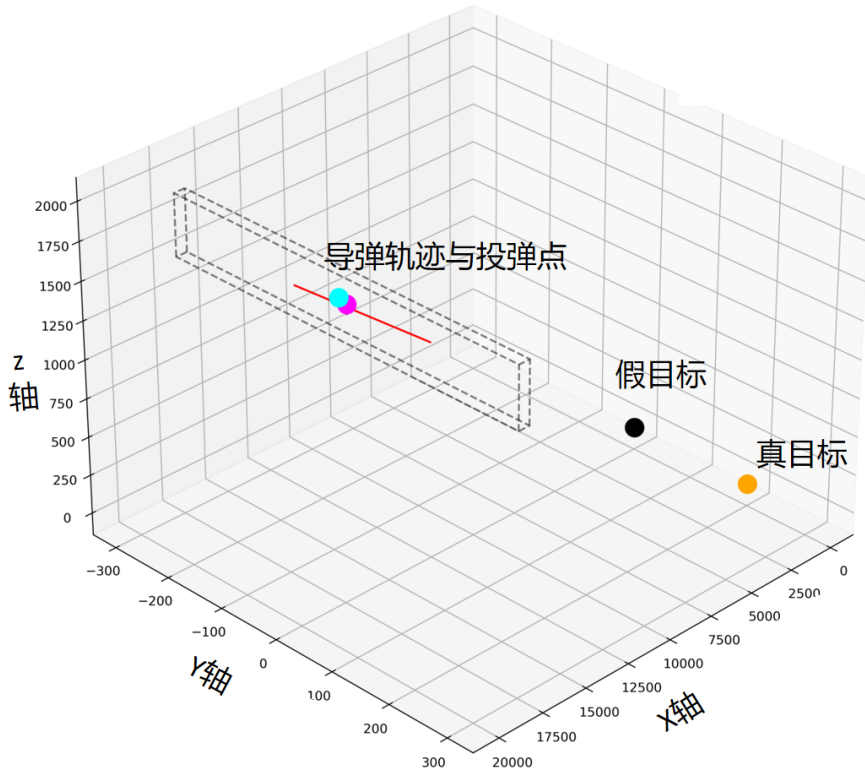


图 1 烟幕弹投放的全过程

动，在 z 轴方向上作匀加速直线运动，则 t_2 时间后爆弹的位置为：

$$F(t_2) = F(t_1) + \widehat{u}_F v_F t_2 + \left(0, 0, -\frac{1}{2} g t_2^2 \right) \quad (4)$$

而第一问已知 $P_{M_0} = (20000, 0, 2000)$ ， $v_m = 300\text{m/s}$ ， $t_1 = 1.5\text{s}$ ， $t_2 = 3.6\text{s}$ ，同时 $\vec{u}_F = (-1, 0, 0)$ ， $v_F = 120\text{m/s}$ ，代入即可计算出起爆时导弹和云团的位置所在。

(2) 得出导弹与云团轨迹

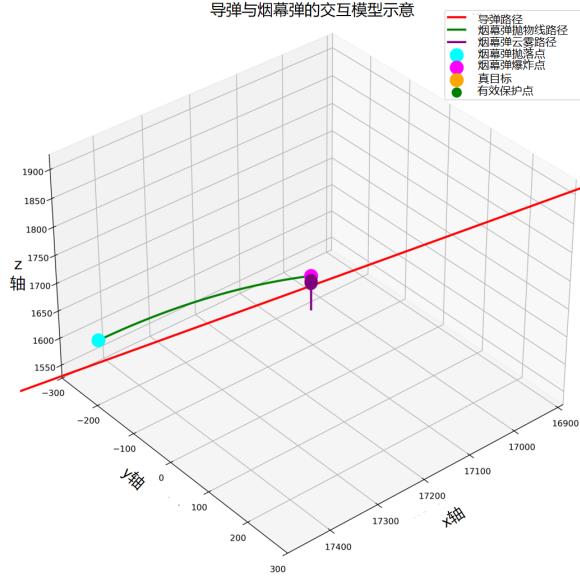
由于导弹的方向大小一直保持不变，则从开始经过时间 t 之后导弹位置随时间的变化函数为：

$$M(t) = P_{M_0} + v_m t \vec{u}_M \quad (5)$$

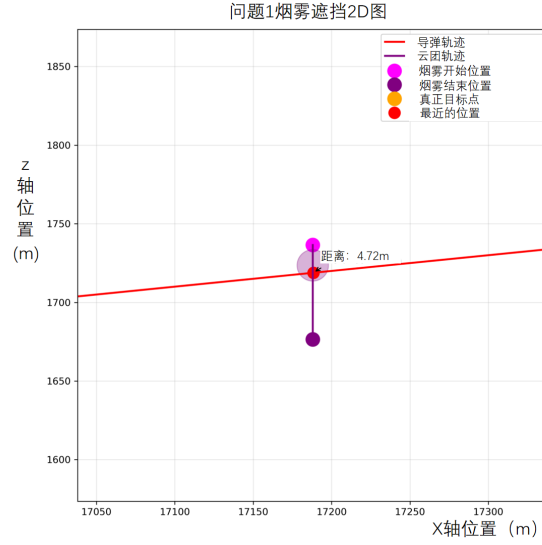
对于云团而言，云团将直接往 z 轴负方向做匀速直线运动，云团运动的方向矢量为 $\widehat{u}_N = (0, 0, -1)$ ，运动大小 $v_N = 3\text{m/s}$ 。云团的运动轨迹为：

$$F(t) = F(t_2) + v_N [t - (t_1 + t_2)] \vec{u}_N \quad (6)$$

所以云团的有效遮蔽球 $B(F(t), R)$ ，其中遮蔽球半径 $R = 10\text{m}$ ，而时间窗口所在区间位于 $[t_1 + t_2, t_1 + t_2 + 20]$ 。



(a) 烟幕弹与导弹交互的三维表征



(b) 问题 2 简化为二维情形

图 2 烟幕弹与导弹交互的二维表征

(3) 借用视线-球体遮挡判定和二分法来计算遮蔽时长

视线-球体遮挡判定定理:

设时刻 t 的导弹位置 $M(t)$ 、目标代表点 P_T 、云团球心 $F(t)$ ，半径为 R 。记视线向量 $a(t) = P_T - M(t)$ ，将球心在该视线上的投影参数

$$\lambda^*(t) = \frac{(F(t) - M(t)) \cdot a(t)}{|a(t)|^2} \quad (7)$$

并作“夹取” $\lambda(t) = \min\{1, \max\{0, \lambda^*(t)\}\}$ 。则视线到球心的最小距离

$$\Delta(t) = |M(t) + \lambda(t)a(t) - F(t)| \quad (8)$$

判定条件：当且仅当

$$\Delta(t) \leq R \text{ 且 } t \in [t_1 + t_2, t_1 + t_2 + 20] \quad (9)$$

成立时，球体对该时刻的导弹-目标视线产生有效遮蔽。

而对于这一题而言， $\Delta(t)$ 的函数可以构造出来， $P_T = (0, 200, 10)$ ，通过二分法对于 $\Delta(t) - R$ 的零点附近求边界点，再联合投影参数落在 $[0, 1]$ 的条件筛选有效片段。

将数据代入位置、并用二分法求解，通过程序得到结果为 1.38s。

4.2 问题二

4.2.1 问题二模型的建立

(1) 二维简化模型与修正方法的存在性

问题二的图例如图 3. 我们观察到，导弹 M1、无人机 FY1、假目标，都落在 xOz 平面上。与此同时，圆柱形固定目标落在 xOz 平面附近。我们先考虑在 xOz 平面上的情况，其后再对模型进行修正。如图 3 所示，我们近似认为烟雾弹看目标的视线是 M1 到原点的直线，很容易证明，如果导弹不会穿过烟雾弹的烟雾，当烟雾弹烟雾球面与视线直线相切时引爆烟雾弹，该烟雾弹此导弹的视线遮挡时间最长，且其起始遮挡和终止遮挡的时刻分别为烟雾弹烟雾球面与直线相切的两不同时刻。由于我们近似认为视线不变，所以在这个不精确的模型中，只要导弹不穿过烟雾弹，对于不同的情形，烟雾弹的遮挡时间保持不变。

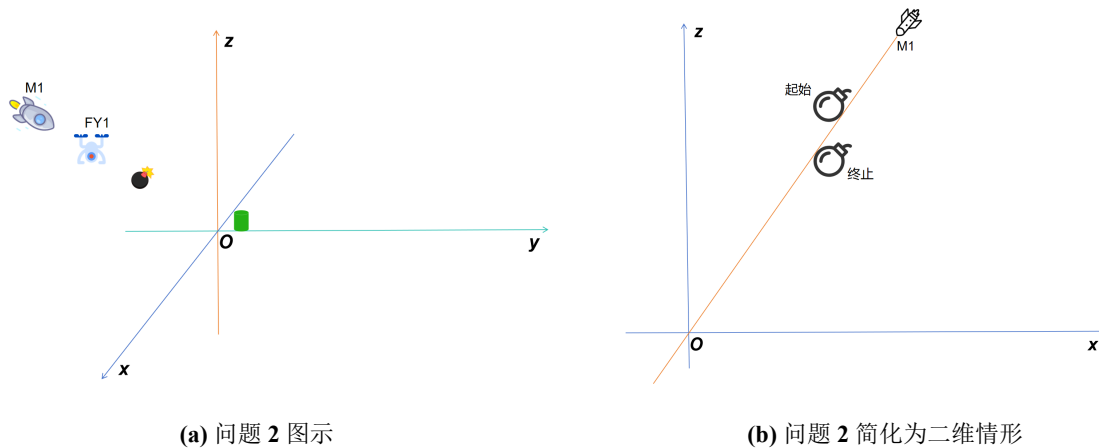


图 3 问题 2 的解析几何图案

然而，这个模型不够精确，得出的结果无法为我们投放烟雾弹的决策提供依据。为此，我们需要修正该模型。该模型的偏差来源于真目标与假目标纵坐标值的偏差。为此，我们先建立一般的数学模型，再借助自主设计的基于此种偏差的迭代算法，得到最终的结果。

我们的算法开始于原点与 M1 之间的连线。烟雾弹开始遮蔽视线的起始点，在暂且不考虑纵坐标偏移的前提下，落在以这条连线为轴心、半径 10m 的圆柱面上。我们首先在该曲面上选择参考点 A。该曲面相对应的运动有三种形式：点的平动、点的旋转动和柱面的锥动。点的平动是指圆柱面上的点平行于轴线的运动，点的旋转动是指圆柱面上的点在平行于底面的圆面上进行的旋转运动，柱面的锥动是指由于开始模型忽略了真目标的纵坐标存在而存在一定的误差，为了消弭这种误差，我们让每个柱面上点的纵坐标都有不同程度的偏移，因此我们不妨直接以原点为锥心向 y 轴正方向旋转柱面轴心，形成柱面的锥动。不难证明，通过这三种运动，存在某种迭代方式，经过足够数量的迭代之后，可以收敛于模型最优点。

(2) 一般方程形式

在给出精确的模型表征前，我们先对问题进行建模。以无人机起始飞行为零时刻，我们首先有烟雾弹起始爆炸点的坐标 (x_0, y_0, z_0) 以及烟雾弹的爆炸时刻 t_3 和烟雾弹终止遮挡的时刻 t_4 。在时刻 t 时，有直线方程

$$L(t) : \frac{x}{20000 - \frac{10}{\sqrt{101}}t} = \frac{y - 200}{-200} = \frac{z}{2000 - \frac{1}{\sqrt{101}}t} \quad (10)$$

有烟雾弹球面方程

$$C(t) : (x - x_0)^2 + (y - y_0)^2 + (z - (z_0 - 3(t - t_3)))^2 = 100, t_3 \leq t \leq t_3 + 20 \quad (11)$$

于是我们有这样的优化问题：

$$\begin{aligned} & \text{maximize} \quad (t_4 - t_3) \\ & \text{s.t.} \quad (1) \ (C(t_3), L(t_3)) \text{ 仅有一解.} \\ & \quad \quad (2) \ (C(t_4), L(t_4)) \text{ 仅有一解.} \end{aligned} \quad (12)$$

经过复杂的计算，我们可以得到，上述第一个条件等价于一个一元二次方程，第二个条件等价于一个一元四次方程（方程具体形式见附录 A）。于是，由两个方程的求根公式 [1] 可知，遮挡时间 $t_1 - t_0$ 是一个由多项式做求根、相加和相除得到的公式。遮挡时间关于 x_0 、 y_0 、 z_0 的偏导是 0，这等价于一个高次多项式等于 0 的方程。由代数基本定理，在圆柱面所在的三维 Euclid 空间内，只有有限个点取到极值。我们观察了附录 A 中给出的解以及其在 MATLAB 中的图像，发现其每个多项式的“波动幅度”都在一个比较小的范围内。于是，对于这样一个模型，我们可以设计出一种泛化性能较好的迭代算法。然而由分析知，迭代算法有掉入局部最优的可能性，这要求我们进行全局寻优的考量。

(3) 最前时间原则

我们已经分析了，真目标纵坐标关于 0 的微小偏移，导致了各种投放方式最大遮挡时间的差异。这种差异体现在空间和时间上。空间上，不同烟雾弹爆炸的位置不同，相对于直线的位置也有差异。时间上，烟雾弹爆炸距离无人机开始运动相隔时长的长短，

也会影响最大遮挡时间，这一点我们会给出一个说明。在这里，时间的影响起着主导作用。事实上，空间因素中烟雾弹与直线的相对位置影响最大，而由于烟雾弹相对本场景极其微小，些许改变无人机速度的大小、方向和飞行时间可以改变水平方向上烟雾弹与直线的相对位置，些许改变下落时间与飞行时间的比例可以改变竖直方向上的相对位置，因此空间并不起很大的影响。

然而，对于时间来说，我们不妨考虑这样一个情形。当导弹距离真目标足够远，我们近似认为导弹与真目标的连线就是其与假目标的连线，于是我们微调烟雾弹的位置，使得烟雾球与底面垂直的最大截面恰好在直线的正上方。烟雾弹下降的过程中，由于导弹与真目标相隔足够远，我们近似认为这条视线依然是之前的那个连线。于是，两次与直线相切都在同一竖直最大截面，高度相差当然最大。然而，假如从无人机启动到烟雾弹爆炸经过的时间足够长，设想第一次球面与直线相切仍在上述位置，第二次相切时，由于导弹向前运动导致视线所在直线向烟雾球的前方偏移，其相切的位置相对球心的高度明显比导弹足够远时的高度低。如果是其他的空间组合，那么前后两次都在差于导弹足够远情况的位置，导致遮挡时间更短。

综上，我们得到**最前时间原则**：要使遮挡 M1 时间尽可能长，需要尽可能让烟雾弹尽可能早地爆炸。

由无人机存在最大速度，我们也不难得到这样一个事实：烟雾弹遮挡 M1 时间最长的位置，不会距离 FY1 起始位置过远。为此我们迭代点的时候，就可以利用这个事实，向 FY1 起始位置迭代即可。

4.2.2 问题二模型的求解

(1) 搜索算法及其结果

我们首先观察以 OM1 为轴心，半径为 10 的圆柱体，如图 4。当 M1 的坐标 (x_m, y_m, z_m) 确定时，该圆柱体上的点的坐标与该点在圆柱体主轴上的投影与原点的距离 λ 和其对应的旋转角度 θ 一一对应。

经过繁杂的计算我们得到，这两类标注方法的转换方程分别为：

$$\begin{cases} x_0 = \frac{\lambda x_m}{L} + 10 (\cos \theta \cdot \hat{u}_{0x} - \sin \theta \cdot \hat{u}_{1x}) \\ y_0 = \frac{\lambda y_m}{L} + 10 (\cos \theta \cdot \hat{u}_{0y} - \sin \theta \cdot \hat{u}_{1y}) \\ z_0 = \frac{\lambda z_m}{L} + 10 (\cos \theta \cdot \hat{u}_{0z} - \sin \theta \cdot \hat{u}_{1z}) \end{cases} \quad (13)$$

$$\begin{cases} \lambda = \frac{x_m x_0 + y_m y_0 + z_m z_0}{L}, \\ \theta = \arctan \left(\frac{\hat{u}_1 \cdot \left[(x_0, y_0, z_0) - \lambda \cdot \frac{(x_m, y_m, z_m)}{L} \right]}{\hat{u}_0 \cdot \left[(x_0, y_0, z_0) - \lambda \cdot \frac{(x_m, y_m, z_m)}{L} \right]} \right) \end{cases} \quad (14)$$

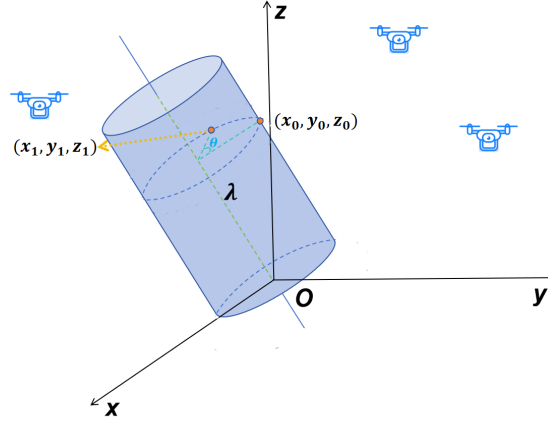


图 4 圆柱体的两类标注

其中 $L = \sqrt{x_m^2 + y_m^2 + z_m^2}$, $\hat{u}_0$ 、 $\hat{u}_1$ 为垂直平面内的单位正交向量, 分别与 0 度基准线和轴线方向一致。

我们提出了一种迭代算法, 该算法需要进行多次迭代。先取圆柱面上远离 FY1 的点, 在每个循环内, 完成点的平动、点的旋转动和柱面的锥动。关于该算法的可行性和优越性, 我们后面做一个说明。

首先先进行点的平动。我们事先设定两个系数, α 和 β 。点可以平行于圆柱面轴心平动, 设定平动系数 s_p 。考虑最前时间原则, 我们先向该方向平行于主轴运动 αs_p , 此时 λ 乘以该系数且 θ 不变。如果对应遮挡时间更长则抵达该点, 如果对应遮挡时间更短则返回原来的位置, 并向反方向运动相同位移探索。如果遮挡时间仍然没有增加, 那么将 α 乘以系数 β 后再重复上述操作。如果在规定的次数内都没有找到, 则停止移动。

接着进行点的旋转。随机从 2,3,5,7 当中选择一个数字 N , 将点所在平行底面的圆面划分成对应片数。我们研究点在圆面上做角度为 $k \cdot \frac{2\pi}{N}$ 的旋转运动后, 所在位置的遮挡时间, 我们选取遮挡时间最长的一个位置去移动点。此时 λ 不变而 θ 增加了对应的角度值。

最后做柱面的锥动。令圆柱顶部圆心向 y 轴正方向移动 s_y , 将点移动相应的位置, 使得点在圆柱主轴上的投影位置与原点的距离不变, 点所在平行底面圆面对应的旋转角不变。此时 λ 和 θ 都不变, 而 (x_m, y_m, z_m) 发生了变化。

数学上, 我们对点的平动和转动都可以先转换成 λ 和 θ 的变换, 再将其数值转变成点的坐标。具体转换的方式已在上文给出。

至于圆柱的旋转, 可以用 Rodrigues 旋转公式。根据该公式, 假设旋转前的坐标是 $\mathbf{x}_{start}$, 旋转后的坐标是 $\mathbf{x}_{final}$, 则有

$$\mathbf{x}_{final} = \mathbf{R}\mathbf{x}_{start} \quad (15)$$

其中 $\mathbf{R} = \begin{pmatrix} \cos \theta + a^2(1 - \cos \theta) & ab(1 - \cos \theta) - c \sin \theta & ac(1 - \cos \theta) + b \sin \theta \\ ba(1 - \cos \theta) + c \sin \theta & \cos \theta + b^2(1 - \cos \theta) & bc(1 - \cos \theta) - a \sin \theta \\ ca(1 - \cos \theta) - b \sin \theta & cb(1 - \cos \theta) + a \sin \theta & \cos \theta + c^2(1 - \cos \theta) \end{pmatrix}$ 。

我们知道，圆柱面上有有限个极值点，且对应遮挡时间的函数值是连续的且震荡不剧烈，那么在极值点附近一定有较为平缓的“高地”。我们用这种方式搜索极值，有利于避免陷入局部最优。另外，“最前时间原则”锚定向无人机起始位置靠拢的全局方向，过滤远离有效区域的伪局部最优，且遮挡时间函数连续低震荡的特性，使不同区域间存在平缓过渡路径。这都保证了最后数据的全局最优性。

同时，本算法避免了在三维空间当中分块采样计算的繁琐性，采用数次循环后，能够精确定位到所希望的区域。如图 5 所示，其等价于一次逼近的情况。模拟实验显示，本算法的时间远少于三维空间的分块采样计算。这更加验证了其独特的优越性。

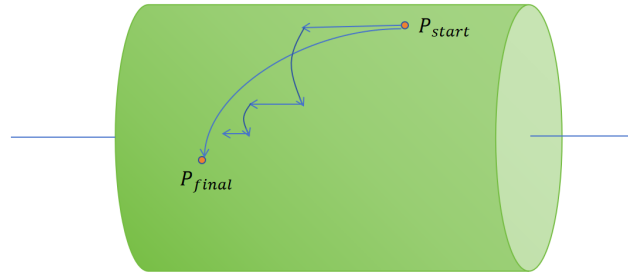


图 5 圆柱体上点的等效迭代

经过数次迭代，我们得出结论：采用搜索算法，其最大遮蔽时间是 4.2s。

(2) 贝叶斯优化算法及其结论

在下文中完成问题四后，我们发现问题二属于问题四的一个子问题。其中，问题二采取明确的模型构建方法，来近似求得问题的解，而问题四采取系统仿真加全局参数优化的方式，针对性的对影响结果的参数进行搜索而不注意物理模型的近似。我们思考：能否将问题四的贝叶斯算法优化来同时解决问题二，验证物理运动模型的准确性。关于贝叶斯优化的具体方法请见章节 4.4。

贝叶斯优化学习参数取无人机飞行方向、飞行速度、投弹时间、起爆延时、起爆点，只需研究一枚烟雾弹对导弹的遮挡时间。而贝叶斯算法架构采用高效树结构 `parzen` 采样器对样本进行采样，并在一定 `step` 之后集中于最优点进行优化，并以一个较低频率探索其他样本（与 `ε-greedy` 算法采样有相似之处）。经验证，问题二的模型得出的最大遮蔽时间为 4.1s，贝叶斯优化得出的最大遮蔽时间是 4.75s。从而我们得出，虽然从可解释性上来看，问题二优于贝叶斯算法，且可以验证正确性，但仍具有一定的局限性。出于模型精确度的考量，我们此题采用数值 4.75s 对应的结果。贝叶斯优化的图表如图 6 所示。

最后，我们得出的结果如图 7 所示。

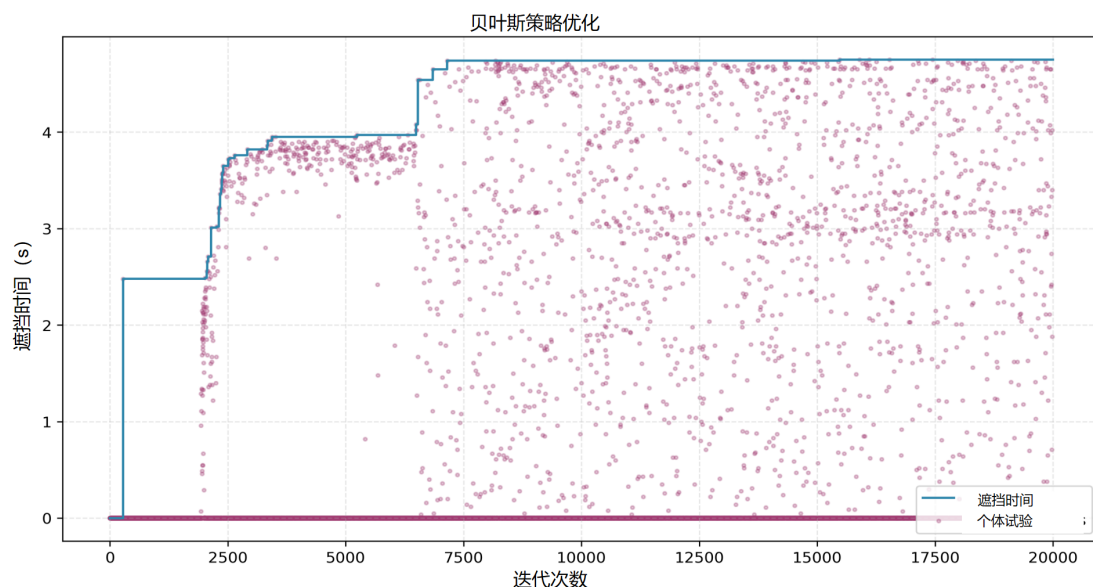


图 6 贝叶斯优化下的收敛图示



图 7 问题二结果

4.3 问题三

模型中有三枚烟雾弹的投放，由于烟雾弹的投放与爆炸时间之间相互影响，同时诸多因素施以限制，本模型具有混沌性，加之第二问求解方式的前提最前时间原则对应一颗烟雾弹，本问题不能单纯利用第二问的思路进行求解。为此，我们采用序列二次规划算法。后面我们将会看到，该算法相对于问题二的算法，更适用于求解多烟雾弹的混沌性问题。

(1) 目标函数

目标为最大化 3 枚烟雾弹的总有效遮蔽时长，采用优化工具 `fmincon` 求解最小值问

题，为此将目标取负，转化为最小值问题， $\min f(x) = -T_{union}(x)$ ，其中 $T_{union}(x)$ 为总有效遮蔽时长，计算步骤为：

1. 对每枚烟雾弹，通过遮蔽判定模型找到有效遮蔽子区间。
2. 收集所有子区间，按起始时间排序，合并重叠 / 相邻区间。
3. 总时长为合并后所有区间的“结束时间 - 起始时间”之和。

(2) 约束条件

线性约束：

1. 无人机飞行速度 v_{d1} ：作为优化变量的第 1 个元素，需满足 $70m/s \leq v_{d1} \leq 140m/s$ 。
2. 无人机飞行方向角 θ_{d1} ：作为优化变量的第 2 个元素，需满足 $-\frac{\pi}{2} \leq \theta_{d1} \leq \frac{\pi}{2}$ 。
 - 该约束限定无人机仅在水平面向目标区域（x 轴负方向两侧）飞行，避免飞行方向偏离有效遮蔽范围，减少无效优化空间。
3. 三枚烟雾弹的投放时间 t_{11}, t_{12}, t_{13} ：分别作为优化变量的第 3、5、7 个元素，每枚弹的投放时间规定需满足 $0.5s \leq t_{1i} \leq 5s (i = 1, 2, 3)$ 。
 - 下界设置 0.5s，目的是避免投放时间过早，导致无人机尚未到达有效投弹位置。
 - 上界设置 5s，目的是避免投放时间过晚，导致引信时延叠加超出爆点时刻上限。
4. 三枚烟雾弹的引信时延 t_{21}, t_{22}, t_{23} ：分别作为优化变量的第 4、6、8 个元素，每枚弹的引信时延规定需满足 $1s \leq t_{2i} \leq 6s$ 。
 - 下界设置 1s，目的是确保烟雾弹有足够时间完成抛体运动。
 - 上界设置 6s，目的是避免引信时延过长，防止爆点时刻超出上限。

非线性约束：

我们有相邻两枚烟雾弹的投放时间差不小于 1s，亦即 $t_{1,i+1} - t_{1i} \leq 1s$ 。具体约束逻辑如下：

- **第 1 个间隔约束：**考虑第 1 枚与第 2 枚烟雾弹的约束式 $t_{12} - t_{11} \geq 1s$ ，矩阵 $\mathbf{A}$ 第 4 行仅将第 1 枚弹投放时间 t_{11} 的系数设为 1，第 2 枚弹投放时间 t_{12} 的系数设为 -1，其余变量系数设为 0；向量 $\mathbf{b}$ 的第 4 个元素为 -1，确保相邻投放间隔不少于 1s。
- **第 2 个间隔约束：**和上面同样的道理，矩阵 $\mathbf{A}$ 第 5 行仅将第 2 枚弹投放时间 t_{12} 的系数设为 1，第 3 枚弹投放时间 t_{13} 的系数设为 -1，其余变量系数设为 0；向量 $\mathbf{b}$ 的第 5 个元素为 -1。

本模型为带约束非线性单目标优化问题，目标函数 $f(\mathbf{x})$ 由轨迹计算、遮蔽判定、区间合并嵌套构成，虽非线性但可微，因此选择 MATLAB 的 `fmincon` 函数作为求解工具，其默认核心算法为序列二次规划（SQP）。

优化变量初始值 $\mathbf{x}_0$ 选择在边界约束内的合理值，避免初始点偏离可行域，设置为 $\mathbf{x}_0 = [120, 0, 0, 3.6, 1, 3.6, 2, 3.6]$ 。迭代次数设置为 12000，目标函数收敛公差为 $1e-6$ ，优

化变量收敛公差为 $1e-6$ 。

根据上述数据，遮挡时间最长的前提下，爆炸时间间隔不必控制在 1s。我们可以发现，问题二中模型所用的最前时间原则具有多目标混沌性，其所要求的最快爆炸亦即爆炸时间相差 1s 在这里并不适用。在这里，我们选取新的算法，从而成功达到了全局最优解。

最后，我们的结果见表 2：

表 2 问题三分析结果

方向 (°)	速度 (m/s)	编号	投放点 x(m)	投放点 y(m)	投放点 z(m)	起爆点 x(m)	起爆点 y(m)	起爆点 z(m)	有效干扰时长 (m)
179.41	107.08	1	17724.36	0.78	1800.0	17342.19	4.7	1737.57	5.97
		2	17555.77	2.51	1800	16241.57	16	1061.77	
		3	17383.6	4.27	1800.0	16881.96	9.42	1692.44	

4.4 问题四

在本问题中，出于模型更加混沌的考量，我们采取了贝叶斯优化算法。该算法采用概率模型对数据进行处理，有利于增加模型的泛化能力与结果的鲁棒性。

贝叶斯优化以目标函数 $f(\mathbf{x})$ 的概率模型为核心，通过迭代更新模型并选择下一个采样点 $\mathbf{x}_{n+1}$ 实现寻优。设优化变量 $\mathbf{x} = [v_{d1}, \theta_{d1}, t_{d1}, \Delta t_{e1}, v_{d2}, \theta_{d2}, t_{d2}, \Delta t_{e2}, v_{d3}, \theta_{d3}, t_{d3}, \Delta t_{e3}]^T$ ，其中 v_{di} 为无人机速度， θ_{di} 为飞行方向角， t_{di} 为投放延迟， Δt_{ei} 为起爆延迟；目标函数 $f(\mathbf{x})$ 为总有效遮蔽时长。

(1) 先验分布推导

假设目标函数 $f(\mathbf{x})$ 服从高斯过程 (GP)：

$$f(\mathbf{x}) \sim \mathcal{GP}(m(\mathbf{x}), k(\mathbf{x}, \mathbf{x}')) \quad (16)$$

其中 $m(\mathbf{x})$ 为均值函数（此处取 $m(\mathbf{x}) = 0$ ）， $k(\mathbf{x}, \mathbf{x}')$ 为协方差函数，采用径向基函数 (RBF)：

$$k(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \exp\left(-\frac{1}{2l^2} \|\mathbf{x} - \mathbf{x}'\|^2\right) + \sigma_n^2 \delta(\mathbf{x}, \mathbf{x}') \quad (17)$$

其中 σ_f^2 为信号方差， l 为长度尺度， σ_n^2 为噪声方差， $\delta(\cdot)$ 为狄拉克函数。

(2) 后验分布推导

设已获取 n 个观测样本 $\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^n$ ，其中 $\mathbf{y}_i = f(\mathbf{x}_i) + \epsilon_i$ ， $\epsilon_i \sim \mathcal{N}(0, \sigma_n^2)$ 。对于新样本 $\mathbf{x}_*$ ，其函数值 $f_* = f(\mathbf{x}_*)$ 与观测值 $\mathbf{y} = [y_1, \dots, y_n]^T$ 的联合分布为：

$$\begin{bmatrix} \mathbf{y} \\ f_* \end{bmatrix} \sim \mathcal{N}\left(\mathbf{0}, \begin{bmatrix} K + \sigma_n^2 I & k_* \\ k_*^T & k_{**} \end{bmatrix}\right) \quad (18)$$

其中 $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$, $k_* = [k(\mathbf{x}_1, \mathbf{x}_*), \dots, k(\mathbf{x}_n, \mathbf{x}_*)]^T$, $k_{**} = k(\mathbf{x}_*, \mathbf{x}_*)$ 。根据高斯分布条件概率公式, f_* 的后验分布为:

$$f_* | \mathcal{D}, \mathbf{x}_* \sim \mathcal{N}(\mu_*(\mathbf{x}_*), \sigma_*^2(\mathbf{x}_*)) \quad (19)$$

其中后验均值为 $\mu_*(\mathbf{x}_*) = k_*^T (K + \sigma_n^2 I)^{-1} \mathbf{y}$, 方差为 $\sigma_*^2(\mathbf{x}_*) = k_{**} - k_*^T (K + \sigma_n^2 I)^{-1} k_*$ 。

(3) 采样函数设计

采用期望改进 (EI) 作为采集函数, 衡量新样本 $\mathbf{x}_*$ 的潜在优化价值:

$$EI(\mathbf{x}_*) = \mathbb{E}[\max(0, f_* - f_{best})] \quad (20)$$

其中 $f_{best} = \max\{y_1, \dots, y_n\}$ 。当 $f_* \sim \mathcal{N}(\mu_*, \sigma_*)$ 时, EI 可解析为:

$$EI(\mathbf{x}_*) = (\mu_* - f_{best}) \Phi\left(\frac{\mu_* - f_{best}}{\sigma_*}\right) + \sigma_* \phi\left(\frac{\mu_* - f_{best}}{\sigma_*}\right) \quad (21)$$

其中 $\Phi(\cdot)$ 、 $\phi(\cdot)$ 分别为标准正态分布的累积分布函数与概率密度函数。每次迭代通过最大化 $EI(\mathbf{x}_*)$ 确定下一个采样点, 直至满足迭代次数停止。

(4) 约束条件融入

针对无人机速度 ($70 \leq v_{di} \leq 140$)、飞行方向 (FY1: $175^\circ \leq \theta_{d1} \leq 185^\circ$, FY2: $180^\circ \leq \theta_{d2} \leq 190^\circ$, FY3: $170^\circ \leq \theta_{d3} \leq 180^\circ$) 及烟幕弹起爆位置 ($p_{sz}^e \geq 0$) 等约束, 采用约束似然修正策略, 对不可行样本赋予 $y_i = 0$ 的观测值, 引导优化过程避开不可行域。

(5) 仿真实现与结果

我们设置迭代次数为 10000 次, 设置采样器为 TPESampler ($seed = 40$), 剪枝策略为 MedianPruner ($n_warmup_steps = 30$), 仿真时间步长为 0.01s, 高斯过程参数为 $\sigma_f^2 = 1$, $l = 1$, $\sigma_n^2 = 0.01$ 。

经贝叶斯优化收敛后, 最优策略参数如下。

结合上述的分析我们发现, 本文构建的贝叶斯优化模型成功融合了物理运动仿真与概率寻优, 通过高斯过程准确建模目标函数的不确定性, 利用期望改进函数高效探索 12 维参数空间, 最终获得了满足题目约束的最优烟幕弹投放策略。该模型相比传统优化方法收敛速度更快, 且能有效处理多约束、多实体协同的复杂干扰任务, 可为无人机烟幕干扰决策提供可靠的理论支撑。

我们得到的结果如表 3 所示:

4.5 问题五

在该模型中, 导弹个数、无人机个数与每个无人机投放的烟雾弹数都是多个, 需要大量的限制条件。在这种情况下, 前面的贝叶斯优化就不再适用。对于这样的多约束问题, 遗传算法 (GA) 是一种很好的选择。因此, 本问题主要采用遗传算法求解。遗传算

表 3 问题三分析结果（单位：m,s）

编号	方向	速度	投放点 x(m)	投放点 y(m)	投放点 z(m)	起爆点 x(m)	起爆点 y(m)	起爆点 z(m)	有效干扰时长 (m)
FY1	177.18	75.71	17799.21	0.04	1800.0	17606.06	9.57	1768.03	4.67
FY2	183.71	120.34	10658.38	1312.90	1400.00	9865.43	1261.42	1186.37	
FY3	178.50	112.57	4028.37	-2948.35	700.0	3528.31	-2935.25	603.25	

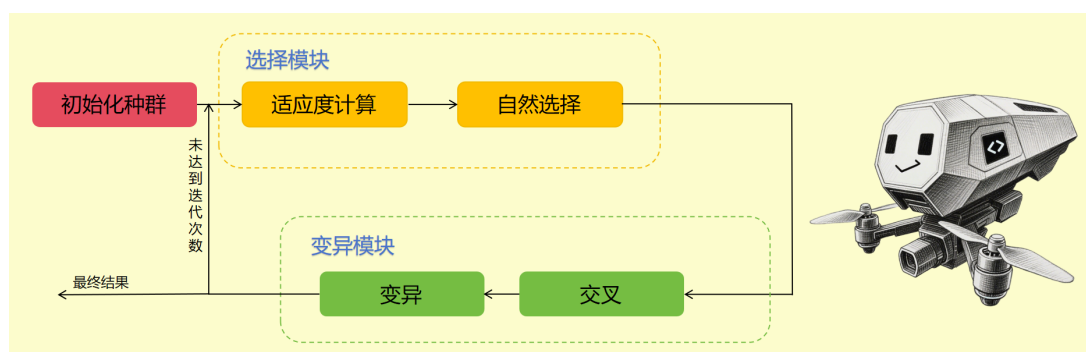


图 8 遗传算法流程图

法是一种模拟自然界遗传变异与自然选择，进而达到全局最优解的算法。其流程如图 8 所示。

我们定义烟雾干扰弹的基因为 $[drone_id, drop_time, explode_delay, velocity, heading_angle]$ ，其中 $drone_id$ 表示无人机编号， $drop_time$ 表示从任务接收起的烟雾弹投放时间， $explode_delay$ 表示从投放到爆炸的延迟， $velocity$ 表示无人机的飞行速度， $heading_angle$ 表示无人机的飞行方向。而一个染色体即为若干个基因的拼接。随后经过实验，我们得到的结果见表 4：

参考文献

- [1] Sergei L Shmakov. A universal method of solving quartic equations. International Journal of Pure and Applied Mathematics, 71(2):251–259, 2011.

表 4 问题五分析结果

无人机编号	运动方向 (°)	速度 (m/s)	弹编号	投放点 x(m)	投放点 y(m)	投放点 z(m)	起爆点 x(m)	起爆点 y(m)	起爆点 z(m)	干扰时长	目标
FY1	198.7	135.1	1	17445.1	0.0	1800.0	16632.3	0.0	1718.4		M1
FY1	198.7	135.1	2	17320.9	0.0	1800.0	16639.1	0.0	1736.9	48.5	M1
FY1	198.7	135.1	3	17202.1	0.0	1800.0	16492.4	0.0	1755.4		M1
FY2	252.3	122.7	1	11521.4	981.6	1400.0	11137.6	726.1	1338.7		M1
FY2	252.3	122.7	2	11589.0	874.2	1400.0	11028.2	617.4	1357.2		M1
FY3	68.0	118.3	1	6701.7	-2136.9	700.0	7168.6	-1845.5	642.1		M2
FY3	68.0	118.3	2	6767.6	-2012.4	700.0	7301.3	-1723.9	623.6	40.3	M2
FY3	68.0	118.3	3	6997.2	-1912.9	700.0	7467.7	-1628.4	605.1		M3
FY4	232.8	126.0	1	10572.6	1629.5	1800.0	9889.0	1432.2	1728.5		M1
FY4	232.8	126.0	2	10025.2	1573.2	1800.0	9714.0	1375.9	1747.0		M1
FY5	140.7	115.9	1	12760.3	-1796.0	1300.0	12428.4	-1545.6	1241.2		M2
FY5	140.7	115.9	2	12193.9	-1681.2	1300.0	12412.7	-1501.0	1261.9	30.7	M3
总投放弹数：12 枚											

附录 A 4.2 中两个运动方程组的解

起始爆炸条件如下：

$$\begin{aligned}
 & (4y_0^2 - 1600y_0 + 160000 - 4x_0^2 - 4z_0^2 - 4y_0^2 + 1600y_0 - 159600)k^2 \\
 & + (-80x_0y_0 - 8y_0z_0 + 16000x_0 + 1600z_0)k + (400x_0^2 + 80x_0z_0 + 4z_0^2 - 404x_0^2 - 404z_0^2 - 404y_0^2 + 161600) \\
 & k = \frac{-b + \sqrt{\Delta}}{2a} \\
 & k = \frac{1}{10 - \frac{1}{200\sqrt{101}}t}
 \end{aligned}
 \tag{22}$$

起始爆炸条件如下：

$$\begin{aligned}
& [(-20x_0 + 1200\sqrt{101})^2 - 4 \times 36360000 \times 101 \times (101 + a^2)]t^4 \\
& + [-2 \times 1200\sqrt{101}(-20x_0 + 1200\sqrt{101}) - 2 \times 1200\sqrt{101} \times 36360000 \times 2 \times 10 \times (101 + a^2) \\
& + 4 \times 1200\sqrt{101}z \times 10 \times 4 \times (101 + a^2)]t^3 \\
& + [1200^2 \times 101 - 2(-20x_0 + 1200\sqrt{101})(-2z_0 + 2(y_0 - 200)a) \\
& - 4(x_0^2 - 100 + (y_0 - 200)^2 + z^2)(101 + a^2) + 4 \times 36360000 \times 101 \times (101 + a^2) \\
& - 8 \times 1200\sqrt{101}z \times (101 + a^2)]t^2 \\
& + [2 \times 1200\sqrt{101}(-2z_0 + 2(y_0 - 200)a) + 8 \times 1200\sqrt{101}z \times (101 + a^2)]t \\
& + (-2z_0 + 2(y_0 - 200)a)^2 - 4(x_0^2 - 100 + (y_0 - 200)^2 + z^2)(101 + a^2) = 0
\end{aligned} \tag{23}$$

附录 B 问题一—matlab 源代码

```

t1=1.5;
t2=3.6;
v1=300;
g=9.8;
daodan1=[20000,0,2000];
norm_daodan1 = norm(daodan1);
unit_vector = daodan1 / norm_daodan1;
daodan2=daodan1-unit_vector*v1*(t1+t2);
drone1=[17800,0,1800];
v2=120;
norm_drone1=norm(drone1);
unit_drone=drone1/norm_drone1;
drone2=drone1-unit_drone*v2*t1;
yanwu1=drone2;
v3x=v2*unit_drone(1,1);
v3z=v2*unit_drone(1,3);
yanwu2=[yanwu1(1,1)+v3x*t2,0,yanwu1(1,3)-(v3z*t2+0.5*g*(t2^2))];
%% 1. 定义云团中心随时间的变化（爆后云团中心运动：c(t) = c0 - 3*(t - t_exp)*[0,0,1]）
c = @(t) c0 - 3*(t - (t1+t2))*[0,0,1];

%% 2. 定义导弹轨迹m(t)（假设导弹M1从(2000, 0, 0)向目标点P_T(0,0,0)运动，速度150m/s）
m0 = [20000,0,2000]; % 导弹初始位置
v_m = unit_vector*v1; % 导弹速度向量
m = @(t) m0 + v_m*t;

%% 3. 定义目标点P_T（假设为原点）
P_T = [0,200,10];

```

```

%% 4. 实现视线-球体遮挡判定函数
delta_fun = @(t) norm(m(t) + lambda_tilde(t).*a_fun(t) - c(t));
a_fun = @(t) P_T - m(t);
lambda_star_fun = @(t) dot(c(t) - m(t), a_fun(t)) / (norm(a_fun(t))^2);
lambda_tilde_fun = @(t) min(1, max(0, lambda_star_fun(t)));

%% 5. 定义判定函数 (Delta(t) <= R)
f = @(t) delta_fun(t) - R;

%% 6. 寻找有效时间区间 (在[t_window_start, t_window_end]内求解f(t)=0的根)

t_range = linspace(t_window_start, t_window_end, 1000);
f_vals = arrayfun(f, t_range);
idx = find(diff(sign(f_vals)) ~= 0);

roots = [];
for i = 1:length(idx)
    t1 = t_range(idx(i));
    t2 = t_range(idx(i)+1);
    root = fzero(f, [t1, t2]);
    roots = [roots, root];
end

if isempty(roots)

    if f(t_window_start) <= 0 && f(t_window_end) <= 0
        effective_time = t_window_end - t_window_start;
    else
        effective_time = 0;
    end
else
    roots = sort(roots);

    intervals = [];
    if f(t_window_start) <= 0
        intervals = [intervals, t_window_start, roots(1)];
    end
    for i = 1:length(roots)-1
        if sign(f((roots(i)+roots(i+1))/2)) <= 0
            intervals = [intervals, roots(i), roots(i+1)];
        end
    end
    if f(t_window_end) <= 0
        intervals = [intervals, roots(end), t_window_end];
    end
end

```

```

end

effective_time = 0;
for i = 1:2:length(intervals)
    effective_time = effective_time + intervals(i+1) - intervals(i);
end
end

fprintf('有效遮蔽时长为: %.2f 秒\n', effective_time)

```

附录 C 问题一—python 源代码

```

import numpy as np

class Missile:
    def __init__(self, initial_pos, target_fake):
        self.speed = 300 # 导弹速度 m/s
        self.initial_pos = np.array(initial_pos, dtype=np.float64)
        self.target_fake = np.array(target_fake, dtype=np.float64)

        # 导弹飞行方向（指向假目标）
        self.direction = (self.target_fake - self.initial_pos) / np.linalg.norm(self.target_fake -
                                          self.initial_pos)
        self.total_distance = np.linalg.norm(self.initial_pos - self.target_fake)
        self.time_to_target = self.total_distance / self.speed # 到达假目标的时间

    def get_position(self, t):
        """计算t时刻导弹的位置（沿直线匀速飞向假目标）"""
        if t >= self.time_to_target:
            return self.target_fake.copy() # 到达后保持在假目标位置
        return self.initial_pos + self.direction * self.speed * t

class Drone:
    def __init__(self, initial_pos, speed, target_dir):
        self.initial_pos = np.array(initial_pos, dtype=np.float64)
        self.speed = speed # 无人机速度 m/s
        # 飞行方向（指向目标，仅水平方向，z坐标不变）
        self.direction = np.array([target_dir[0] - initial_pos[0], target_dir[1] - initial_pos[1],
                                    0])
        self.direction = self.direction / np.linalg.norm(self.direction) # 归一化方向向量

    def get_position(self, t):
        """计算t时刻无人机的位置（水平匀速直线运动）"""
        return self.initial_pos + self.direction * self.speed * t

```

```

def get_velocity_vector(self):
    """返回无人机的速度向量（用于烟雾弹惯性运动）"""
    return self.direction * self.speed

class SmokeBomb:
    def __init__(self, drop_pos, drop_time, drone_velocity):
        self.drop_pos = np.array(drop_pos, dtype=np.float64) # 投放位置
        self.drop_time = drop_time # 投放时间（相对于任务开始）
        self.drone_velocity = drone_velocity # 投放瞬间无人机的速度向量（xy方向）
        self.explode_time = None # 起爆时间
        self.explode_pos = None # 起爆位置（含xy惯性运动和z自由落体）
        self.g = 9.8 # 重力加速度 m/s²
        self.sink_speed = 3 # 起爆后下沉速度 m/s

    def calculate_explode_position(self, explode_delay):
        """计算从投放到起爆的运动：xy惯性匀速 + z自由落体"""
        self.explode_time = self.drop_time + explode_delay # 起爆时间
        time_in_air = explode_delay # 投放后到起爆前的飞行时间

        # 1. xy方向：保持无人机速度做匀速运动
        xy_displacement = self.drone_velocity * time_in_air
        explode_xy = self.drop_pos[:2] + xy_displacement[:2]

        # 2. z方向：自由落体运动（不低于地面）
        z_fall = 0.5 * self.g * (time_in_air ** 2)
        explode_z = max(0, self.drop_pos[2] - z_fall)

        # 起爆位置（z固定，后续仅下沉）
        self.explode_pos = np.array([explode_xy[0], explode_xy[1], explode_z])
        return self.explode_pos

    def get_position(self, t):
        """计算t时刻烟雾云团的位置"""
        if t < self.explode_time:
            return None # 未起爆，不参与遮蔽
        # 起爆后：xy位置固定，z方向匀速下沉
        sink_time = t - self.explode_time
        current_z = max(0, self.explode_pos[2] - self.sink_speed * sink_time)
        return np.array([self.explode_pos[0], self.explode_pos[1], current_z])

    def is_shielding(self, missile_pos, smoke_pos, target_real, shield_radius=10):
        """
        判断烟雾是否有效遮蔽真目标：
        1. 烟雾必须位于导弹和真目标之间的连线上（投影在0~1之间）

```



```

2. 烟雾到连线的距离 10m (有效遮蔽半径)
"""
# 向量定义
missile_to_target = target_real - missile_pos # 导弹→真目标
missile_to_smoke = smoke_pos - missile_pos # 导弹→烟雾

# 导弹到真目标的距离 (避免除以0)
dist_missile_target = np.linalg.norm(missile_to_target)
if dist_missile_target < 1e-6:
    return False # 导弹已到达真目标, 遮蔽无意义

# 烟雾在"导弹→真目标"方向上的投影比例 (0=导弹位置, 1=真目标位置)
projection_ratio = np.dot(missile_to_smoke, missile_to_target) / (dist_missile_target ** 2)

# 条件1: 烟雾必须在导弹和真目标之间
if not (0 < projection_ratio < 1):
    return False

# 条件2: 烟雾到连线的距离 10m
closest_point_on_line = missile_pos + projection_ratio * missile_to_target # 连线上最近点
dist_smoke_line = np.linalg.norm(smoke_pos - closest_point_on_line)
return dist_smoke_line <= shield_radius

# 问题1 主仿真
if __name__ == "__main__":
    # 1. 固定参数定义
    target_fake = np.array([0, 0, 0]) # 假目标位置 (原点)
    target_real = np.array([0, 200, 0]) # 真目标位置 (题目给定)
    drop_delay = 1.5 # 任务开始后1.5s投放
    explode_delay = 3.6 # 投放后3.6s起爆
    drone_speed = 120 # 无人机速度120m/s (题目给定)

    # 2. 初始化导弹M1和无人机FY1
    missile_M1 = Missile(initial_pos=[20000, 0, 2000], target_fake=target_fake)
    drone_FY1 = Drone(initial_pos=[17800, 0, 1800], speed=drone_speed, target_dir=target_fake)

    # 3. 计算投放状态 (投放时间、位置、无人机速度向量)
    drop_time = drop_delay
    drop_pos = drone_FY1.get_position(drop_time) # 投放位置
    drone_velocity = drone_FY1.get_velocity_vector() # 投放时无人机速度向量

    # 4. 初始化烟雾弹并计算起爆位置
    smoke_bomb = SmokeBomb(drop_pos, drop_time, drone_velocity)
    explode_pos = smoke_bomb.calculate_explode_position(explode_delay)

    # 5. 仿真参数设置

```

```

time_step = 0.00001 # 时间步长（精度自定义）
# 仿真结束条件：导弹到达假目标 或 烟幕失效（起爆后20s）
max_simulation_time = min(missile_M1.time_to_target, smoke_bomb.explode_time + 20)
current_time = 0
shielding_duration = 0 # 有效遮蔽时长累计

# 6. 打印关键初始信息
print("=" * 50)
print(f"投放时间: {drop_time:.1f}s | 投放位置: {np.round(drop_pos, 2)}")
print(f"起爆时间: {smoke_bomb.explode_time:.1f}s | 起爆位置: {np.round(explode_pos, 2)}")
print(f"导弹到达假目标时间: {missile_M1.time_to_target:.1f}s | 仿真结束时间: {max_simulation_time:.1f}s")
print("=" * 50)

# 7. 逐时刻仿真
while current_time <= max_simulation_time:
    # 获取当前时刻各物体位置
    missile_pos = missile_M1.get_position(current_time)
    smoke_pos = smoke_bomb.get_position(current_time)

    # 检查是否满足遮蔽条件
    if (smoke_pos is not None and # 烟幕已起爆
        current_time <= smoke_bomb.explode_time + 20 and # 烟幕未失效
        is_shielding(missile_pos, smoke_pos, target_real)): # 有效遮蔽
        shielding_duration += time_step

    # 时间步进
    current_time += time_step

# 8. 输出结果
print(f"\n最终有效遮蔽时长: {shielding_duration:.5f}秒")
print("=" * 50)

```

附录 D 问题二-python 源代码 1

```

import numpy as np
import optuna
import matplotlib.pyplot as plt

# ----- 复用并微调基础类 -----
class Missile:
    def __init__(self, initial_pos, target_fake):
        self.speed = 300
        self.initial_pos = np.array(initial_pos, dtype=np.float64)
        self.target_fake = np.array(target_fake, dtype=np.float64)

```

```

self.direction = (self.target_fake - self.initial_pos) / np.linalg.norm(self.target_fake
    - self.initial_pos)
self.total_distance = np.linalg.norm(self.initial_pos - self.target_fake)
self.time_to_target = self.total_distance / self.speed

def get_position(self, t):
    if t >= self.time_to_target:
        return self.target_fake.copy()
    return self.initial_pos + self.direction * self.speed * t

class Drone:
    def __init__(self, initial_pos, speed, target_dir):
        """微调: target_dir直接传入单位方向向量（不再计算）"""
        self.initial_pos = np.array(initial_pos, dtype=np.float64)
        self.speed = speed
        self.direction = np.array(target_dir, dtype=np.float64)
        self.direction /= np.linalg.norm(self.direction) # 确保归一化

    def get_position(self, t):
        return self.initial_pos + self.direction * self.speed * t

    def get_velocity_vector(self):
        return self.direction * self.speed

class SmokeBomb:
    def __init__(self, drop_pos, drop_time, drone_velocity):
        self.drop_pos = np.array(drop_pos, dtype=np.float64)
        self.drop_time = drop_time
        self.drone_velocity = drone_velocity
        self.explode_time = None
        self.explode_pos = None
        self.g = 9.8
        self.sink_speed = 3

    def calculate_explode_position(self, explode_delay):
        self.explode_time = self.drop_time + explode_delay
        time_in_air = explode_delay

        # xy方向惯性运动
        xy_displacement = self.drone_velocity * time_in_air
        explode_xy = self.drop_pos[:2] + xy_displacement[:2]

        # z方向自由落体（若落地则抛出异常，Optuna会剪枝该解）
        z_fall = 0.5 * self.g * (time_in_air ** 2)
        explode_z = self.drop_pos[2] - z_fall

```

```

if explode_z < 0:
    raise ValueError("Smoke bomb hits ground before explosion")

self.explode_pos = np.array([explode_xy[0], explode_xy[1], explode_z])
return self.explode_pos

def get_position(self, t):
    if t < self.explode_time:
        return None
    sink_time = t - self.explode_time
    current_z = max(0, self.explode_pos[2] - self.sink_speed * sink_time)
    return np.array([self.explode_pos[0], self.explode_pos[1], current_z])

def is_shielding(missile_pos, smoke_pos, target_real, shield_radius=10):
    missile_to_target = target_real - missile_pos
    missile_to_smoke = smoke_pos - missile_pos
    dist_missile_target = np.linalg.norm(missile_to_target)

    if dist_missile_target < 1e-6:
        return False

    projection_ratio = np.dot(missile_to_smoke, missile_to_target) / (dist_missile_target **
        2)
    if not (0 < projection_ratio < 1):
        return False

    closest_point = missile_pos + projection_ratio * missile_to_target
    dist_smoke_line = np.linalg.norm(smoke_pos - closest_point)
    return dist_smoke_line <= shield_radius

# ----- 定义Optuna目标函数 -----
def objective(trial):
    """Optuna接收trial对象，用于采样决策变量"""
    # 1. 采样决策变量 (Optuna自动搜索最优值)
    v = trial.suggest_float("velocity", 70.0, 140.0) # 无人机速度: 70-140m/s
    theta = trial.suggest_float("direction_angle", 0.0, 360.0) # 飞行方向: 0-360度
    t_drop = trial.suggest_float("drop_delay", 0.0, 70.0) #
        投放延迟: 0-70s (小于导弹飞行时间74s)
    t_explode = trial.suggest_float("explode_delay", 0.1, 18.0) # 起爆延迟: 0.1-18s (避免落地)

    # 2. 解析飞行方向为单位向量
    dir_x = np.cos(np.radians(theta))
    dir_y = np.sin(np.radians(theta))
    drone_dir = [dir_x, dir_y, 0.0]

```

```

# 3. 初始化仿真对象
target_fake = [0, 0, 0]
target_real = [0, 200, 0]
drone = Drone(initial_pos=[17800, 0, 1800], speed=v, target_dir=drone_dir)
missile = Missile(initial_pos=[20000, 0, 2000], target_fake=target_fake)

# 4. 计算投放与起爆状态（若无效则返回0）
try:
    drop_pos = drone.get_position(t_drop)
    drone_vel = drone.get_velocity_vector()
    smoke = SmokeBomb(drop_pos, t_drop, drone_vel)
    smoke.calculate_explode_position(t_explode)
except Exception as e:
    return 0.0 # 无效解, 返回0

# 5. 高效仿真: 记录遮蔽起止时间
max_t = min(missile.time_to_target, smoke.explode_time + 20)
current_t = 0.0
time_step = 0.01
shield_time = 0.0
shield_start = None

while current_t <= max_t:
    missile_pos = missile.get_position(current_t)
    smoke_pos = smoke.get_position(current_t)

    if smoke_pos is not None and current_t <= smoke.explode_time + 20:
        if is_shielding(missile_pos, smoke_pos, target_real):
            if shield_start is None:
                shield_start = current_t # 开始遮蔽
            else:
                if shield_start is not None:
                    shield_time += (current_t - shield_start) # 结束遮蔽, 累加时长
                shield_start = None
            current_t += time_step

# 处理仿真结束时仍在遮蔽的情况
if shield_start is not None:
    shield_time += (max_t - shield_start)

return shield_time # 返回目标函数值（遮蔽时长）

# ----- 运行贝叶斯优化 -----
if __name__ == "__main__":
    # 1. 配置优化参数
    study = optuna.create_study(

```

```

direction="maximize", # 目标: 最大化遮蔽时长
sampler=optuna.samplers.TPESampler(), # 采样器: 树结构Parzen估计器 (高效)
pruner=optuna.pruners.MedianPruner(n_warmup_steps=10) # 剪枝: 提前终止无效解
)

# 2. 运行优化 (迭代5000次, 可根据时间调整)
study.optimize(objective, n_trials=20000, show_progress_bar=True)

# 3. 输出最优结果
print("\n" + "="*60)
print("最优投放策略: ")
best_trial = study.best_trial
print(f"1. 无人机速度: {best_trial.params['velocity']:.2f} m/s")
print(f"2. 飞行方向: {best_trial.params['direction_angle']:.2f} ° (xy平面) ")
print(f"3. 投放延迟: {best_trial.params['drop_delay']:.2f} s")
print(f"4. 起爆延迟: {best_trial.params['explode_delay']:.2f} s")

# 4. 计算最优策略的详细状态 (投放点、起爆点)
v = best_trial.params['velocity']
theta = best_trial.params['direction_angle']
t_drop = best_trial.params['drop_delay']
t_explode = best_trial.params['explode_delay']

dir_x = np.cos(np.radians(theta))
dir_y = np.sin(np.radians(theta))
drone = Drone([17800, 0, 1800], v, [dir_x, dir_y, 0])
drop_pos = drone.get_position(t_drop)
drone_vel = drone.get_velocity_vector()
smoke = SmokeBomb(drop_pos, t_drop, drone_vel)
explode_pos = smoke.calculate_explode_position(t_explode)

print(f"5. 投放点坐标: ({drop_pos[0]:.2f}, {drop_pos[1]:.2f}, {drop_pos[2]:.2f}) m")
print(f"6. 起爆点坐标: ({explode_pos[0]:.2f}, {explode_pos[1]:.2f}, {explode_pos[2]:.2f}) m")
print(f"7. 最大有效遮蔽时长: {best_trial.value:.2f} s")
print("="*60)

# 提取所有trial的迭代序号和对应的遮蔽时长
trial_numbers = [trial.number for trial in study.trials]
trial_values = [trial.value for trial in study.trials]

# 计算"随迭代次数的当前最长时间" (累积最大值)
max_values = []
current_max = 0.0
for val in trial_values:
    if val > current_max:
        current_max = val

```

```

max_values.append(current_max)

# 绘制曲线
plt.figure(figsize=(10, 6))
plt.plot(trial_numbers, max_values, color='#2E86AB', linewidth=1.5, label='shield time')
plt.scatter(trial_numbers, trial_values, color='#A23B72', s=5, alpha=0.3,
            label='individual trials')

# 添加图表元素
plt.title('Varied Delay Strategy Optimization', fontsize=14, pad=20)
plt.xlabel('Iteration', fontsize=12, labelpad=10)
plt.ylabel('Explosion Time (s)', fontsize=12, labelpad=10)
plt.legend(loc='lower right', fontsize=10)
plt.grid(True, alpha=0.3, linestyle='--')

# 调整布局并保存 (dpi=300保证清晰度)
plt.tight_layout()
plt.savefig('q2.png', dpi=300, bbox_inches='tight')
plt.close() # 关闭画布, 释放内存
print("\n 最长时间随迭代次数变化曲线图已保存为 q2.png")

```

附录 E 问题二—python 源代码 2

```

import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

class SmokeGrenadeOptimizer:
    def __init__(self):
        # 导弹初始位置
        self.M1_start = np.array([20000, 0, 2000])

        # 假目标位置 (原点)
        self.decoy = np.array([0, 0, 0])

        # 真目标位置
        self.real_target = np.array([0, 200, 0])

        # 无人机FY1初始位置
        self.FY1_start = np.array([17800, 0, 1800])

        # 导弹速度 (300 m/s)
        self.missile_speed = 300

```

```

# 计算导弹方向向量（指向假目标）
missile_dir = self.decoy - self.M1_start
self.missile_dir = missile_dir / np.linalg.norm(missile_dir)

# 无人机速度（假设为最大速度140 m/s）
self.drone_speed = 140

# 烟雾弹参数
self.smoke_radius = 10 # 有效半径
self.smoke_duration = 20 # 有效持续时间
self.smoke_sink_speed = 3 # 下沉速度

# 重力加速度
self.g = 9.8

# 迭代参数
self.max_iterations = 500
self.alpha = 50 # 平动系数
self.beta = 0.8 # 学习率衰减系数

# 计算导弹到达目标的大致时间
missile_to_decoy = np.linalg.norm(self.M1_start - self.decoy)
self.missile_arrival_time = missile_to_decoy / self.missile_speed

print(f"导弹到达假目标的大致时间: {self.missile_arrival_time:.2f} s")

# 圆柱轴线（导弹到假目标的连线）
self.cylinder_axis = self.decoy - self.M1_start
self.cylinder_axis = self.cylinder_axis / np.linalg.norm(self.cylinder_axis)

# 初始点选择（在圆柱面上靠近导弹-真目标视线）
self.current_point = self.find_initial_point_near_line_of_sight()

# 记录最佳解
self.best_point = self.current_point.copy()
self.best_occlusion_time = 0
self.best_t0 = 0

def find_initial_point_near_line_of_sight(self):
    """找到靠近导弹-真目标视线的初始点"""
    # 计算导弹到真目标的视线
    missile_to_target = self.real_target - self.M1_start
    line_dir = missile_to_target / np.linalg.norm(missile_to_target)

    # 在视线上选择一个点，距离导弹约1/3处
    t_param = 0.3 # 参数，选择视线上的点
    point_on_line = self.M1_start + t_param * missile_to_target

```



```

# 计算该点到圆柱轴线的投影
vec_to_axis = point_on_line - self.M1_start
projection = np.dot(vec_to_axis, self.cylinder_axis) * self.cylinder_axis
perpendicular = vec_to_axis - projection

# 计算垂直分量, 确保点在圆柱面上
if np.linalg.norm(perpendicular) > 0:
    perpendicular = perpendicular / np.linalg.norm(perpendicular) * self.smoke_radius

# 初始点
initial_point = self.M1_start + projection + perpendicular

print(f"初始点位置: {initial_point}")
return initial_point

def missile_position(self, t):
    """计算导弹在时刻t的位置"""
    # 导弹以300m/s的速度向假目标飞行
    return self.M1_start + self.missile_dir * self.missile_speed * t

def line_of_sight(self, t):
    """计算时刻t的视线方程(从导弹到真实目标)"""
    missile_pos = self.missile_position(t)
    return missile_pos, self.real_target - missile_pos

def smoke_center(self, t, t0, smoke_point):
    """计算烟雾弹在时刻t的球心位置"""
    # 烟雾弹投放位置 + 下落位移
    # 注意: 烟雾弹在重力作用下下落, 然后起爆后匀速下沉

    # 计算自由落体时间
    if t <= t0:
        # 投放后到起爆前: 自由落体
        fall_time = t
        z_pos = smoke_point[2] - 0.5 * self.g * fall_time ** 2
        return np.array([smoke_point[0], smoke_point[1], z_pos])
    else:
        # 起爆后: 匀速下沉
        # 先计算起爆时刻的位置
        burst_z = smoke_point[2] - 0.5 * self.g * t0 ** 2
        burst_pos = np.array([smoke_point[0], smoke_point[1], burst_z])

        # 计算起爆后的下沉
        sink_time = t - t0
        z_pos = burst_pos[2] - self.smoke_sink_speed * sink_time
        return np.array([burst_pos[0], burst_pos[1], z_pos])

```

```

def distance_to_line(self, point, line_point, line_dir):
    """计算点到直线的距离"""
    line_dir_norm = line_dir / np.linalg.norm(line_dir)
    vec = point - line_point
    projection = np.dot(vec, line_dir_norm) * line_dir_norm
    return np.linalg.norm(vec - projection)

def calculate_occlusion_time(self, smoke_point, t0):
    """计算遮挡时间"""
    # 确保起爆时间在导弹到达之前
    t0 = min(t0, self.missile_arrival_time - 1)

    # 计算遮挡时间
    start_time = t0
    end_time = min(t0 + self.smoke_duration, self.missile_arrival_time)

    # 使用密集采样方法计算遮挡时间
    sample_times = np.linspace(start_time, end_time, 200) # 增加采样点数量
    occlusion_mask = []

    for t in sample_times:
        smoke_center_pos = self.smoke_center(t, t0, smoke_point)
        line_point, line_dir = self.line_of_sight(t)
        distance = self.distance_to_line(smoke_center_pos, line_point, line_dir)
        occlusion_mask.append(distance <= self.smoke_radius)

    # 计算连续遮挡的时间段
    occlusion_intervals = []
    in_occlusion = False
    start_occlusion = 0

    for i, occluded in enumerate(occlusion_mask):
        if occluded and not in_occlusion:
            in_occlusion = True
            start_occlusion = sample_times[i]
        elif not occluded and in_occlusion:
            in_occlusion = False
            occlusion_intervals.append((start_occlusion, sample_times[i]))

    if in_occlusion:
        occlusion_intervals.append((start_occlusion, end_time))

    # 计算总遮挡时间
    total_occlusion_time = sum(end - start for start, end in occlusion_intervals)

    return total_occlusion_time

```

```

def translation_motion(self, point, alpha):
    """点的平动：沿着圆柱轴线方向移动"""
    # 计算点到轴线的投影
    vec_to_point = point - self.M1_start
    projection = np.dot(vec_to_point, self.cylinder_axis) * self.cylinder_axis
    perpendicular = vec_to_point - projection

    # 平动方向：向FY1起始位置移动
    direction_to_fy1 = self.FY1_start - point
    if np.linalg.norm(direction_to_fy1) > 0:
        direction_to_fy1 = direction_to_fy1 / np.linalg.norm(direction_to_fy1)

    # 尝试平动
    new_point = point + direction_to_fy1 * alpha

    # 确保点在圆柱面上
    vec_to_new_point = new_point - self.M1_start
    projection = np.dot(vec_to_new_point, self.cylinder_axis) * self.cylinder_axis
    perpendicular = vec_to_new_point - projection
    if np.linalg.norm(perpendicular) > 0:
        perpendicular = perpendicular / np.linalg.norm(perpendicular) * self.smoke_radius

    new_point = self.M1_start + projection + perpendicular

    return new_point

def rotation_motion(self, point, N):
    """点的旋转：在圆柱圆周方向上旋转"""
    best_point = point
    best_occlusion = 0

    # 计算点在圆柱上的角度
    vec_to_point = point - self.M1_start
    projection = np.dot(vec_to_point, self.cylinder_axis) * self.cylinder_axis
    perpendicular = vec_to_point - projection

    if np.linalg.norm(perpendicular) > 0:
        # 计算当前角度
        # 选择一个参考向量（垂直于圆柱轴线）
        ref_vector = np.array([1, 0, 0]) # 任意参考向量
        if np.abs(np.dot(ref_vector, self.cylinder_axis)) > 0.9:
            ref_vector = np.array([0, 1, 0]) # 如果与轴线太接近，换一个

        # 使参考向量垂直于轴线
        ref_vector = ref_vector - np.dot(ref_vector, self.cylinder_axis) * self.cylinder_axis
        if np.linalg.norm(ref_vector) > 0:

```

```

ref_vector = ref_vector / np.linalg.norm(ref_vector)

# 计算当前角度
current_angle = np.arctan2(
    np.dot(perpendicular, np.cross(self.cylinder_axis, ref_vector)),
    np.dot(perpendicular, ref_vector)
)

# 尝试N个不同的角度
for k in range(N):
    angle = current_angle + k * 2 * np.pi / N

# 计算新位置
new_perpendicular = (
    np.cos(angle) * ref_vector +
    np.sin(angle) * np.cross(self.cylinder_axis, ref_vector)
) * self.smoke_radius

new_point = self.M1_start + projection + new_perpendicular

# 计算遮挡时间
occlusion_time = self.calculate_occlusion_time(new_point, 0)

if occlusion_time > best_occlusion:
    best_occlusion = occlusion_time
    best_point = new_point

return best_point

def cone_motion(self, point, s_y):
    """柱面的锥动：调整圆柱轴线方向"""
    # 以原点为锥心向y轴正方向旋转柱面轴心
    cone_center = self.decoy

    # 计算当前轴线与y轴的夹角
    current_axis = self.cylinder_axis
    y_axis = np.array([0, 1, 0])

    # 计算旋转轴（当前轴线与y轴的叉积）
    rotation_axis = np.cross(current_axis, y_axis)
    if np.linalg.norm(rotation_axis) > 0:
        rotation_axis = rotation_axis / np.linalg.norm(rotation_axis)

    # 计算旋转角度（根据s_y调整）
    angle = s_y * 0.01 # 小角度旋转

    # 旋转矩阵

```

```

cos_angle = np.cos(angle)
sin_angle = np.sin(angle)
one_minus_cos = 1 - cos_angle

R = np.array([
    [cos_angle + rotation_axis[0] ** 2 * one_minus_cos,
     rotation_axis[0] * rotation_axis[1] * one_minus_cos - rotation_axis[2] * sin_angle,
     rotation_axis[0] * rotation_axis[2] * one_minus_cos + rotation_axis[1] * sin_angle],

    [rotation_axis[1] * rotation_axis[0] * one_minus_cos + rotation_axis[2] * sin_angle,
     cos_angle + rotation_axis[1] ** 2 * one_minus_cos,
     rotation_axis[1] * rotation_axis[2] * one_minus_cos - rotation_axis[0] * sin_angle],

    [rotation_axis[2] * rotation_axis[0] * one_minus_cos - rotation_axis[1] * sin_angle,
     rotation_axis[2] * rotation_axis[1] * one_minus_cos + rotation_axis[0] * sin_angle,
     cos_angle + rotation_axis[2] ** 2 * one_minus_cos]
])

# 旋转轴线
new_axis = np.dot(R, current_axis)
self.cylinder_axis = new_axis / np.linalg.norm(new_axis)

# 调整点的位置，保持点在圆柱上的相对位置不变
vec_to_point = point - cone_center
projection = np.dot(vec_to_point, current_axis) * current_axis
perpendicular = vec_to_point - projection

# 计算点在圆柱上的角度
if np.linalg.norm(perpendicular) > 0:
    # 选择一个参考向量
    ref_vector = np.array([1, 0, 0])
    if np.abs(np.dot(ref_vector, current_axis)) > 0.9:
        ref_vector = np.array([0, 1, 0])

    ref_vector = ref_vector - np.dot(ref_vector, current_axis) * current_axis
    if np.linalg.norm(ref_vector) > 0:
        ref_vector = ref_vector / np.linalg.norm(ref_vector)

    current_angle = np.arctan2(
        np.dot(perpendicular, np.cross(current_axis, ref_vector)),
        np.dot(perpendicular, ref_vector)
    )

# 计算新位置
new_ref_vector = np.array([1, 0, 0])
if np.abs(np.dot(new_ref_vector, self.cylinder_axis)) > 0.9:
    new_ref_vector = np.array([0, 1, 0])

```

```

new_ref_vector = new_ref_vector - np.dot(new_ref_vector, self.cylinder_axis) *
    self.cylinder_axis
if np.linalg.norm(new_ref_vector) > 0:
new_ref_vector = new_ref_vector / np.linalg.norm(new_ref_vector)

new_perpendicular = (
np.cos(current_angle) * new_ref_vector +
np.sin(current_angle) * np.cross(self.cylinder_axis, new_ref_vector)
) * self.smoke_radius

new_point = cone_center + projection + new_perpendicular
return new_point

return point

def optimize_t0(self, smoke_point):
    """优化烟雾弹起爆时间"""
    best_t0 = 0
    best_occlusion = 0

    # 尝试多个起爆时间，从0到导弹到达时间-1秒
    t0_options = np.linspace(0, self.missile_arrival_time - 1, 50)

    for t0 in t0_options:
        occlusion_time = self.calculate_occlusion_time(smoke_point, t0)
        if occlusion_time > best_occlusion:
            best_occlusion = occlusion_time
            best_t0 = t0

    return best_t0, best_occlusion

def optimize(self):
    """执行优化算法"""
    print("开始迭代优化...")
    print(f"初始点: {self.current_point}")

    # 记录迭代过程
    history = []

    # 初始优化起爆时间
    best_t0, best_occlusion = self.optimize_t0(self.current_point)
    self.best_t0 = best_t0
    self.best_occlusion_time = best_occlusion

    # 迭代优化
    for iteration in range(self.max_iterations):

```

```

# 平动
new_point_trans = self.translation_motion(self.current_point, self.alpha)
t0_trans, occlusion_time_trans = self.optimize_t0(new_point_trans)

# 如果平动后遮挡时间增加, 则接受新位置
if occlusion_time_trans > self.best_occlusion_time:
    self.current_point = new_point_trans
    self.best_occlusion_time = occlusion_time_trans
    self.best_t0 = t0_trans
    self.best_point = self.current_point.copy()
else:
    # 尝试反方向平动
    new_point_trans_rev = self.translation_motion(self.current_point, -self.alpha)
    t0_trans_rev, occlusion_time_trans_rev = self.optimize_t0(new_point_trans_rev)

    if occlusion_time_trans_rev > self.best_occlusion_time:
        self.current_point = new_point_trans_rev
        self.best_occlusion_time = occlusion_time_trans_rev
        self.best_t0 = t0_trans_rev
        self.best_point = self.current_point.copy()
    else:
        # 减少步长
        self.alpha *= self.beta

# 旋转
N = np.random.choice([2, 3, 5, 7]) # 随机选择N值
new_point_rot = self.rotation_motion(self.current_point, N)
t0_rot, occlusion_time_rot = self.optimize_t0(new_point_rot)

if occlusion_time_rot > self.best_occlusion_time:
    self.current_point = new_point_rot
    self.best_occlusion_time = occlusion_time_rot
    self.best_t0 = t0_rot
    self.best_point = self.current_point.copy()

# 锥动
s_y = 0.1 # 锥动系数
new_point_cone = self.cone_motion(self.current_point, s_y)
t0_cone, occlusion_time_cone = self.optimize_t0(new_point_cone)

if occlusion_time_cone > self.best_occlusion_time:
    self.current_point = new_point_cone
    self.best_occlusion_time = occlusion_time_cone
    self.best_t0 = t0_cone
    self.best_point = self.current_point.copy()

# 记录迭代历史

```

```

history.append({
    'iteration': iteration,
    'point': self.current_point.copy(),
    'occlusion_time': self.best_occlusion_time,
    't0': self.best_t0,
    'alpha': self.alpha
})

# 每50次迭代输出一次进度
if iteration % 50 == 0:
    print(f"Iteration {iteration}: Occlusion time = {self.best_occlusion_time:.4f} s, "
          f"t0 = {self.best_t0:.2f} s, Alpha = {self.alpha:.4f}")

    print(f"优化完成! 最佳遮挡时间: {self.best_occlusion_time:.4f} s")
    print(f"最佳烟雾弹位置: {self.best_point}")
    print(f"最佳起爆时间: {self.best_t0:.2f} s")

# 验证结果
self.validate_result()

return history

def validate_result(self):
    """验证优化结果, 确保计算正确"""
    print("\n验证优化结果:")

    # 计算导弹到达假目标的时间
    missile_to_decoy = np.linalg.norm(self.M1_start - self.decoy)
    missile_arrival_time = missile_to_decoy / self.missile_speed
    print(f"导弹到达假目标的时间: {missile_arrival_time:.2f} s")

    # 计算导弹到真目标的视线长度
    missile_to_target = np.linalg.norm(self.M1_start - self.real_target)
    print(f"导弹到真目标的距离: {missile_to_target:.2f} m")

    # 计算烟雾弹的最佳位置
    print(f"烟雾弹最佳位置: {self.best_point}")

    # 计算烟雾弹到导弹-真目标视线的距离
    missile_pos = self.missile_position(self.best_t0)
    line_dir = self.real_target - missile_pos
    distance = self.distance_to_line(self.best_point, missile_pos, line_dir)
    print(f"烟雾弹到视线的距离: {distance:.2f} m")

    # 计算烟雾弹的下降高度
    fall_height = 0.5 * self.g * self.best_t0 ** 2
    smoke_height = self.best_point[2] - fall_height

```



```

print(f"烟雾弹起爆时的高度: {smoke_height:.2f} m")

# 计算烟雾弹的有效遮挡时间
# 烟雾弹下沉速度3m/s, 有效半径10m
# 烟雾弹需要覆盖视线, 因此需要计算烟雾弹下沉过程中覆盖视线的时间

# 计算视线与烟雾弹的夹角
missile_pos = self.missile_position(self.best_t0)
line_dir = self.real_target - missile_pos
line_dir_norm = line_dir / np.linalg.norm(line_dir)

smoke_to_line = self.best_point - missile_pos
projection = np.dot(smoke_to_line, line_dir_norm) * line_dir_norm
perpendicular = smoke_to_line - projection

# 计算烟雾弹下沉过程中覆盖视线的时间
# 烟雾弹下沉速度3m/s, 有效半径10m
# 烟雾弹需要下沉的距离为烟雾弹到视线的距离
sink_distance = np.linalg.norm(perpendicular)
sink_time = sink_distance / self.smoke_sink_speed if self.smoke_sink_speed > 0 else 0

print(f"烟雾弹需要下沉的距离: {sink_distance:.2f} m")
print(f"烟雾弹下沉所需时间: {sink_time:.2f} s")

# 计算理论最大遮挡时间
theoretical_occlusion = min(sink_time, self.smoke_duration)
print(f"理论最大遮挡时间: {theoretical_occlusion:.2f} s")

# 比较计算结果与理论值
print(f"计算遮挡时间: {self.best_occlusion_time:.4f} s")
print(f"理论遮挡时间: {theoretical_occlusion:.2f} s")
print(f"差异: {abs(self.best_occlusion_time - theoretical_occlusion):.4f} s")

def visualize(self, history):
    """可视化优化过程"""
    # 提取历史数据
    iterations = [h['iteration'] for h in history]
    occlusion_times = [h['occlusion_time'] for h in history]
    points = [h['point'] for h in history]
    t0s = [h['t0'] for h in history]
    alphas = [h['alpha'] for h in history]

    # 创建图表
    fig = plt.figure(figsize=(15, 10))

    # 遮挡时间随迭代次数的变化
    ax1 = fig.add_subplot(221)

```

```

ax1.plot(iterations, occlusion_times, 'b-')
ax1.set_xlabel('Iteration')
ax1.set_ylabel('Occlusion Time (s)')
ax1.set_title('Occlusion Time vs Iteration')
ax1.grid(True)

# 起爆时间随迭代次数的变化
ax2 = fig.add_subplot(222)
ax2.plot(iterations, t0s, 'g-')
ax2.set_xlabel('Iteration')
ax2.set_ylabel('Burst Time (s)')
ax2.set_title('Burst Time vs Iteration')
ax2.grid(True)

# 平动系数随迭代次数的变化
ax3 = fig.add_subplot(223)
ax3.plot(iterations, alphas, 'r-')
ax3.set_xlabel('Iteration')
ax3.set_ylabel('Alpha')
ax3.set_title('Translation Coefficient vs Iteration')
ax3.grid(True)

# 烟雾弹位置在XY平面的投影
ax4 = fig.add_subplot(224)
x_coords = [p[0] for p in points]
y_coords = [p[1] for p in points]
ax4.scatter(x_coords, y_coords, c=iterations, cmap='viridis')
ax4.plot(x_coords, y_coords, 'k--', alpha=0.3)
ax4.scatter(self.FY1_start[0], self.FY1_start[1], c='r', marker='o', s=100, label='FY1
    Start')
ax4.scatter(self.real_target[0], self.real_target[1], c='g', marker='s', s=100,
    label='Real Target')
ax4.scatter(self.decoy[0], self.decoy[1], c='r', marker='x', s=100, label='Decoy')

# 绘制导弹轨迹在XY平面的投影
missile_traj = np.array([self.missile_position(t)[:2] for t in np.linspace(0,
    self.missile_arrival_time, 50)])
ax4.plot(missile_traj[:, 0], missile_traj[:, 1], 'm-', alpha=0.5, label='Missile
    Trajectory')

ax4.set_xlabel('X')
ax4.set_ylabel('Y')
ax4.set_title('Smoke Grenade Position (XY Projection)')
ax4.legend()
ax4.grid(True)

plt.tight_layout()

```

```

plt.show()

# 3D轨迹图
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')

# 绘制烟雾弹位置轨迹
x_coords = [p[0] for p in points]
y_coords = [p[1] for p in points]
z_coords = [p[2] for p in points]
ax.plot(x_coords, y_coords, z_coords, 'b-', alpha=0.5)
ax.scatter(x_coords, y_coords, z_coords, c=iterations, cmap='viridis', s=20)

# 标记关键点
ax.scatter(self.FY1_start[0], self.FY1_start[1], self.FY1_start[2],
c='r', marker='o', s=100, label='FY1 Start')
ax.scatter(self.real_target[0], self.real_target[1], self.real_target[2],
c='g', marker='s', s=100, label='Real Target')
ax.scatter(self.decoy[0], self.decoy[1], self.decoy[2],
c='r', marker='x', s=100, label='Decoy')

# 标记最佳点
ax.scatter(self.best_point[0], self.best_point[1], self.best_point[2],
c='m', marker='*', s=200, label='Best Point')

# 绘制导弹轨迹
missile_traj = np.array([self.missile_position(t) for t in np.linspace(0,
self.missile_arrival_time, 50)])
ax.plot(missile_traj[:, 0], missile_traj[:, 1], missile_traj[:, 2],
'm-', alpha=0.5, label='Missile Trajectory')

ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
ax.set_title('Smoke Grenade Position (3D)')
ax.legend()

plt.show()

# 执行优化
optimizer = SmokeGrenadeOptimizer()
history = optimizer.optimize()
optimizer.visualize(history)

```

附录 F 问题三-matlab 源代码

```
%% 1. 固定基础参数（已知量）
v1 = 300;           % 导弹速度 (m/s)
g = 9.8;           % 重力加速度 (m/s^2)
R = 10;            % 烟幕云团半径 (m)
t_window_total = 20; % 单枚烟幕有效时间窗口长度 (s)
daodan1 = [20000, 0, 2000]; % 导弹初始位置 (m)
drone1 = [17800, 0, 1800]; % 无人机初始位置 (m)
P_T = [0, 200, 10]; % 目标点 (m)
t_exp_max = 10;     % 爆点时刻上限 (s)
n_bombs = 3;        % 烟雾弹数量
t_interval_min = 1; % 投放间隔最小值 (s)

%% 2. 读取原始文档信息（统一转为细胞数组，避免维度不匹配）
original_filename = 'result1(1).xlsx';
original_data_cell = {}; % 原始数据（细胞数组）
original_notes_cell = {'注：以x轴为正向，逆时针方向为正，取值0~360（度）。', '', '', '',
    '', '', '', '', '', ''};

try

[~, ~, raw_cell] = xlsread(original_filename, 'Sheet1');
fprintf('成功读取原始文档: %s\n', original_filename);
if size(raw_cell, 1) >= 2 % 确保有数据行
    original_data_cell = raw_cell(2:min(4, size(raw_cell, 1)), 1:10);
    if size(original_data_cell, 2) < 10
        original_data_cell = [original_data_cell, cell(size(original_data_cell, 1), 10 -
            size(original_data_cell, 2))];
    end
end
if size(raw_cell, 1) >= 5
    original_notes_cell = raw_cell(5:5, 1:10)
    if size(original_notes_cell, 2) < 10
        original_notes_cell = [original_notes_cell, cell(1, 10 - size(original_notes_cell, 2))];
    end
end
catch
    original_data_cell = cell(3, 10);
    fprintf('原始文档不存在，将创建包含原始格式+优化结果的新文档\n');
end

%% 3. 定义非线性规划的目标函数：输入共享参数+3枚投放参数，输出负总遮蔽时长（并集）
% 未知参数向量 x = [v2, theta, t1_1, t2_1, t1_2, t2_2, t1_3, t2_3]
obj_fun = @(x) -calculate_union_time(x, v1, g, R, t_window_total, ...
    daodan1, drone1, P_T, t_exp_max, n_bombs);
```

```

%% 4. 非线性规划参数设置
x0 = [120, 0, 0, 3.6, 1, 3.6, 2, 3.6]; % 初始值

lb = [70, -pi/2, 0.5, 1, 0.5, 1, 0.5, 1]; % 下界
ub = [140, pi/2, 5, 6, 5, 6, 5, 6]; % 上界

% 线性约束：爆点时刻+投放间隔（5个约束）
A = zeros(5, 8);
A(1, [3, 4]) = [1, 1]; % 第1枚爆点时刻约束
A(2, [5, 6]) = [1, 1]; % 第2枚爆点时刻约束
A(3, [7, 8]) = [1, 1]; % 第3枚爆点时刻约束
A(4, [3, 5]) = [1, -1]; % 第2-1枚投放间隔约束
A(5, [5, 7]) = [1, -1]; % 第3-2枚投放间隔约束
b = [t_exp_max, t_exp_max, t_exp_max, -t_interval_min, -t_interval_min];
options = optimoptions('fmincon', ...
    'Display', 'iter', ...
    'MaxFunctionEvaluations', 12000, ...
    'MaxIterations', 6000, ...
    'TolFun', 1e-6, ...
    'TolX', 1e-6);

%% 5. 调用fmincon求解
[x_opt, fval, exitflag, output] = fmincon(obj_fun, x0, A, b, [], [], lb, ub, [], options);

%% 6. 解析结果（计算投放点/起爆点/有效时长）
v2_opt = x_opt(1);
theta_opt = x_opt(2);
theta_opt_deg = rad2deg(theta_opt);
unit_vector_drone = [cos(theta_opt), sin(theta_opt), 0]; % 无人机运动单位向量

param = struct();
for i = 1:n_bombs
    idx = 2 + (i-1)*2 + 1:2 + 2*i;
    param(i).t1 = x_opt(idx(1)); % 投放时间
    param(i).t2 = x_opt(idx(2)); % 引信时延
    param(i).t_exp = param(i).t1 + param(i).t2; % 爆点时刻
    param(i).launch_x = drone1(1) - unit_vector_drone(1) * v2_opt * param(i).t1;
    param(i).launch_y = drone1(2) - unit_vector_drone(2) * v2_opt * param(i).t1;
    param(i).launch_z = drone1(3) - unit_vector_drone(3) * v2_opt * param(i).t1;
    smoke_horiz_x = unit_vector_drone(1) * v2_opt * param(i).t2;
    smoke_horiz_y = unit_vector_drone(2) * v2_opt * param(i).t2;
    smoke_vert_z = 0.5 * g * param(i).t2^2; % 重力下落位移
    param(i).explode_x = param(i).launch_x - smoke_horiz_x;
    param(i).explode_y = param(i).launch_y - smoke_horiz_y;
    param(i).explode_z = param(i).launch_z - smoke_vert_z;
    x_single = [param(i).t1, param(i).t2, v2_opt, theta_opt];

```

```

intervals_single = calculate_single_intervals(x_single, v1, g, R, t_window_total, ...
daodan1, drone1, P_T, t_exp_max, unit_vector_drone);

if ~isempty(intervals_single)
param(i).effective_time = sum(intervals_single(:,2) - intervals_single(:,1));
else
param(i).effective_time = 0;
end
end

total_union_time = -fval; % 三枚并集总遮蔽时长

%% 7. 输出结果到命令窗口
fprintf('\n===== 三枚烟雾弹并集优化结果 =====\n');
fprintf('无人机共享参数: 速度=%.4f m/s, 方向角=%.4f°\n\n', v2_opt, theta_opt_deg);

for i = 1:n_bombs
fprintf('第%d枚: 投放时间=%.4fs, 引信时延=%.4fs, 爆点=%.4fs, 单枚有效时长=%.4fs\n', ...
i, param(i).t1, param(i).t2, param(i).t_exp, param(i).effective_time);
if i < n_bombs
fprintf('与下一枚投放间隔=%.4fs\n\n', param(i+1).t1 - param(i).t1);
end
end

fprintf('总遮蔽时长 (并集)=%.6fs\n', total_union_time);
fprintf('收敛标志= %d, 迭代次数= %d\n', exitflag, output.iterations);
fprintf('===== \n');

%% 8. 整合原始数据与优化结果 (统一细胞数组类型, 确保维度一致)
standard_headers = {
    '无人机运动方向', '无人机运动速度 (m/s)', '烟幕干扰弹编号', ...
    '烟幕干扰弹投放点的x坐标 (m)', '烟幕干扰弹投放点的y坐标 (m)', '烟幕干扰弹投放点的z坐标 (m)', ...
    '烟幕干扰弹起爆点的x坐标 (m)', '烟幕干扰弹起爆点的y坐标 (m)', '烟幕干扰弹起爆点的z坐标 (m)', ...
    '有效干扰时长 (s)'};
};
optimized_data = {
    % 第1枚烟雾弹优化结果
    num2str(theta_opt_deg, '%.4f'), num2str(v2_opt, '%.4f'), 1, ...
    num2str(param(1).launch_x, '%.2f'), num2str(param(1).launch_y, '%.2f'),
        num2str(param(1).launch_z, '%.2f'), ...
    num2str(param(1).explode_x, '%.2f'), num2str(param(1).explode_y, '%.2f'),
        num2str(param(1).explode_z, '%.2f'), ...
    num2str(param(1).effective_time, '%.4f');
    % 第2枚烟雾弹优化结果
    num2str(theta_opt_deg, '%.4f'), num2str(v2_opt, '%.4f'), 2, ...

```

```

num2str(param(2).launch_x, '%.2f'), num2str(param(2).launch_y, '%.2f'),
    num2str(param(2).launch_z, '%.2f'), ...
num2str(param(2).explode_x, '%.2f'), num2str(param(2).explode_y, '%.2f'),
    num2str(param(2).explode_z, '%.2f'), ...
num2str(param(2).effective_time, '%.4f');
% 第3枚烟雾弹优化结果
num2str(theta_opt_deg, '%.4f'), num2str(v2_opt, '%.4f'), 3, ...
num2str(param(3).launch_x, '%.2f'), num2str(param(3).launch_y, '%.2f'),
    num2str(param(3).launch_z, '%.2f'), ...
num2str(param(3).explode_x, '%.2f'), num2str(param(3).explode_y, '%.2f'),
    num2str(param(3).explode_z, '%.2f'), ...
num2str(param(3).effective_time, '%.4f')
};
separator_row = {'---原始数据结束---', '', '', '', '', '', '', '', ''};
optimize_label_row = {'---优化计算结果---', '', '', '', '', '', '', '', ''};
full_data = [
standard_headers;           % 第1行：列名（与原始文档一致）
original_data_cell;         % 第2-4行：原始空数据（保留格式）
separator_row;              % 第5行：分隔符
optimize_label_row;         % 第6行：优化结果标签
optimized_data;             % 第7-9行：优化结果数据
original_notes_cell         % 第10行：原始备注（保留）
];

%% 9. 写入Excel文件（包含原始信息+优化结果）
output_filename = 'result1(1).xlsx';
xlswrite(output_filename, full_data, 'Sheet1', 'A1');
total_info = {
    '总结果说明', '数值', '单位';
    '原始文档来源', original_filename, '';
    '无人机运动方向（优化值）', num2str(theta_opt_deg, '%.4f'), '度';
    '无人机运动速度（优化值）', num2str(v2_opt, '%.4f'), 'm/s';
    '三枚烟雾弹总遮蔽时长（并集）', num2str(total_union_time, '%.6f'), 's';
    '优化收敛标志', num2str(exitflag), '1=收敛, -1=未收敛';
    '优化迭代次数', num2str(output.iterations), '次';
    '单枚烟幕有效时间窗口', num2str(t_window_total), 's';
    '爆点时刻上限', num2str(t_exp_max), 's'
};
xlswrite(output_filename, total_info, '总结果说明', 'A1');
fprintf('\n结果已成功输出到Excel文件: %s\n', output_filename);
fprintf('文件包含以下内容 (Sheet1) : \n');
fprintf('1. 第1行：原始文档列名（完全匹配）\n');
fprintf('2. 第2-4行：原始烟雾弹空数据（保留格式）\n');
fprintf('3. 第5行：原始数据与优化结果分隔符\n');
fprintf('4. 第6-9行：优化计算结果（3枚烟雾弹详细参数）\n');
fprintf('5. 第10行：原始文档备注（保留）\n');
fprintf('6. 新增"总结果说明"工作表：全局优化信息\n');

```

附录 G 问题三-python 源代码

```
import numpy as np
import optuna
import pandas as pd

# ----- 基础类定义 -----
class Missile:
    def __init__(self, initial_pos, target_fake):
        self.speed = 300
        self.initial_pos = np.array(initial_pos, dtype=np.float64)
        self.target_fake = np.array(target_fake, dtype=np.float64)
        self.direction = (self.target_fake - self.initial_pos) /
            np.linalg.norm(self.target_fake - self.initial_pos)
        self.total_distance = np.linalg.norm(self.initial_pos - self.target_fake)
        self.time_to_target = self.total_distance / self.speed

    def get_position(self, t):
        if t >= self.time_to_target:
            return self.target_fake.copy()
        return self.initial_pos + self.direction * self.speed * t

class Drone:
    def __init__(self, initial_pos, speed, target_dir):
        """target_dir传入单位方向向量"""
        self.initial_pos = np.array(initial_pos, dtype=np.float64)
        self.speed = speed
        self.direction = np.array(target_dir, dtype=np.float64)
        self.direction /= np.linalg.norm(self.direction) # 确保归一化

    def get_position(self, t):
        return self.initial_pos + self.direction * self.speed * t

    def get_velocity_vector(self):
        return self.direction * self.speed

class SmokeBomb:
    def __init__(self, drop_pos, drop_time, drone_velocity):
        self.drop_pos = np.array(drop_pos, dtype=np.float64)
        self.drop_time = drop_time
        self.drone_velocity = drone_velocity
```



```

self.explode_time = None
self.explode_pos = None
self.g = 9.8
self.sink_speed = 3

def calculate_explode_position(self, explode_delay):
    self.explode_time = self.drop_time + explode_delay
    time_in_air = explode_delay

    # xy方向惯性运动
    xy_displacement = self.drone_velocity * time_in_air
    explode_xy = self.drop_pos[:2] + xy_displacement[:2]

    # z方向自由落体（若落地则抛出异常）
    z_fall = 0.5 * self.g * (time_in_air ** 2)
    explode_z = self.drop_pos[2] - z_fall
    if explode_z < 0:
        raise ValueError("Smoke bomb hits ground before explosion")

    self.explode_pos = np.array([explode_xy[0], explode_xy[1], explode_z])
    return self.explode_pos

def get_position(self, t):
    if t < self.explode_time:
        return None
    sink_time = t - self.explode_time
    current_z = max(0, self.explode_pos[2] - self.sink_speed * sink_time)
    return np.array([self.explode_pos[0], self.explode_pos[1], current_z])

# ----- 辅助函数 -----
def is_shielding(missile_pos, smoke_pos, target_real, shield_radius=10):
    missile_to_target = target_real - missile_pos
    missile_to_smoke = smoke_pos - missile_pos
    dist_missile_target = np.linalg.norm(missile_to_target)

    if dist_missile_target < 1e-6:
        return False

    projection_ratio = np.dot(missile_to_smoke, missile_to_target) / (dist_missile_target
        ** 2)
    if not (0 < projection_ratio < 1):
        return False

    closest_point = missile_pos + projection_ratio * missile_to_target
    dist_smoke_line = np.linalg.norm(smoke_pos - closest_point)
    return dist_smoke_line <= shield_radius

```

```

def merge_intervals(intervals):
    """合并重叠或相邻的时间区间，返回并集时长"""
    if not intervals:
        return 0.0

    # 按开始时间排序
    sorted_intervals = sorted(intervals, key=lambda x: x[0])
    merged = [sorted_intervals[0]]

    # 合并重叠区间
    for current in sorted_intervals[1:]:
        last = merged[-1]
        if current[0] <= last[1]: # 重叠或相邻，合并
            new_start = last[0]
            new_end = max(last[1], current[1])
            merged[-1] = (new_start, new_end)
        else: # 不重叠，新增区间
            merged.append(current)

    # 计算总时长
    total = sum(end - start for start, end in merged)
    return total

# ----- 目标函数 -----
def objective_3bombs(trial):
    """优化目标：3枚烟幕弹的遮蔽时间并集最大化，约束投放间隔 1s"""
    # 1. 采样无人机参数（3枚弹共享）
    v = trial.suggest_float("drone_velocity", 70.0, 140.0) # 无人机速度
    theta = trial.suggest_float("drone_direction", 175.0, 185.0) # 方向聚焦180°±5°提高效率
    dir_x = np.cos(np.radians(theta))
    dir_y = np.sin(np.radians(theta))
    drone_dir = [dir_x, dir_y, 0.0] # 飞行方向向量

    # 2. 采样3枚弹的参数（带间隔约束）
    t_drop1 = trial.suggest_float("t_drop1", 0.0, 25.0)
    t_explode1 = trial.suggest_float("t_explode1", 0.1, 18.0)

    t_drop2 = trial.suggest_float("t_drop2", t_drop1 + 1.0, 28.0) # 至少间隔1s
    t_explode2 = trial.suggest_float("t_explode2", 0.1, 18.0)

    t_drop3 = trial.suggest_float("t_drop3", t_drop2 + 1.0, 31.0) # 至少间隔1s
    t_explode3 = trial.suggest_float("t_explode3", 0.1, 18.0)

    # 3. 初始化仿真对象

```

```

target_fake = [0, 0, 0]
target_real = [0, 200, 0]
drone = Drone(initial_pos=[17800, 0, 1800], speed=v, target_dir=drone_dir)
missile = Missile(initial_pos=[20000, 0, 2000], target_fake=target_fake)

# 4. 计算3枚弹的投放/起爆状态
bombs = []
try:
    # 第1枚弹
    drop_pos1 = drone.get_position(t_drop1)
    vel1 = drone.get_velocity_vector()
    bomb1 = SmokeBomb(drop_pos1, t_drop1, vel1)
    bomb1.calculate_explode_position(t_explode1)
    bombs.append(bomb1)

    # 第2枚弹
    drop_pos2 = drone.get_position(t_drop2)
    vel2 = drone.get_velocity_vector()
    bomb2 = SmokeBomb(drop_pos2, t_drop2, vel2)
    bomb2.calculate_explode_position(t_explode2)
    bombs.append(bomb2)

    # 第3枚弹
    drop_pos3 = drone.get_position(t_drop3)
    vel3 = drone.get_velocity_vector()
    bomb3 = SmokeBomb(drop_pos3, t_drop3, vel3)
    bomb3.calculate_explode_position(t_explode3)
    bombs.append(bomb3)
except Exception as e:
    return 0.0 # 任意弹无效, 返回0

# 5. 仿真3枚弹的遮蔽区间
max_simulation_t = min(missile.time_to_target, max(b.explode_time + 20 for b in bombs))
current_t = 0.0
time_step = 0.01 # 仿真步长
bomb_intervals = [[], [], []] # 存储3枚弹的遮蔽区间: (start, end)

while current_t <= max_simulation_t:
    missile_pos = missile.get_position(current_t)

    # 遍历3枚弹, 记录各自的遮蔽区间
    for i, bomb in enumerate(bombs):
        smoke_pos = bomb.get_position(current_t)
        if smoke_pos is None or current_t > bomb.explode_time + 20:
            continue # 未起爆或已失效

    # 判断是否遮蔽

```

```

if is_shielding(missile_pos, smoke_pos, target_real):
    # 若当前未记录开始时间，记录
    if not bomb_intervals[i] or bomb_intervals[i][-1][1] != float('inf'):
        bomb_intervals[i].append((current_t, float('inf')))
    else:
        # 若当前处于遮蔽中，记录结束时间
        if bomb_intervals[i] and bomb_intervals[i][-1][1] == float('inf'):
            bomb_intervals[i][-1] = (bomb_intervals[i][-1][0], current_t)

current_t += time_step

# 处理仿真结束时仍在遮蔽的情况
for i in range(3):
    if bomb_intervals[i] and bomb_intervals[i][-1][1] == float('inf'):
        bomb_intervals[i][-1] = (bomb_intervals[i][-1][0], max_simulation_t)

# 6. 合并3枚弹的遮蔽区间，计算并集时长
all_intervals = []
for intervals in bomb_intervals:
    all_intervals.extend(intervals)
total_shield = merge_intervals(all_intervals)

return total_shield # 返回并集时长作为目标值

# ----- 主函数 -----
if __name__ == "__main__":
    # 1. 配置优化
    study = optuna.create_study(
        direction="maximize",
        sampler=optuna.samplers.TPESampler(seed=41), # 固定种子，结果可复现
        pruner=optuna.pruners.MedianPruner(n_warmup_steps=30) # 热身20次后剪枝
    )

    # 2. 运行优化（3枚弹仿真耗时更长，迭代2000次）
    study.optimize(objective_3bombs, n_trials=3000, show_progress_bar=True)

    # 3. 解析最优参数
    best = study.best_trial
    v = best.params["drone_velocity"]
    theta = best.params["drone_direction"]
    dir_x = np.cos(np.radians(theta))
    dir_y = np.sin(np.radians(theta))
    drone_dir = [dir_x, dir_y, 0.0]
    drone = Drone([17800, 0, 1800], v, drone_dir)

    # 4. 计算3枚弹的详细状态

```

```

bombs_info = []
for i in [1,2,3]:
    t_drop = best.params[f"t_drop{i}"]
    t_explode = best.params[f"t_explode{i}"]
    drop_pos = drone.get_position(t_drop)
    vel = drone.get_velocity_vector()
    bomb = SmokeBomb(drop_pos, t_drop, vel)
    explode_pos = bomb.calculate_explode_position(t_explode)
    bombs_info.append({
        "t_drop": t_drop, "t_explode": t_explode,
        "drop_pos": drop_pos, "explode_pos": explode_pos
    })

# 5. 输出结果
print("\n" + "="*80)
print("最优3枚烟幕弹投放策略（间隔 1s）")
print("="*80)
print(f"无人机参数：速度={v:.2f}m/s, 方向={theta:.2f}°")
print(f"总遮蔽时间并集：{best.value:.2f}s")
print("-"*80)
for i, info in enumerate(bombs_info, 1):
    print(f"第{i}枚弹：")
    print(f"  投放延迟：{info['t_drop']:.2f}s | 投放位置：({info['drop_pos'][0]:.2f},
        {info['drop_pos'][1]:.2f}, {info['drop_pos'][2]:.2f})m")
    print(f"  起爆延迟：{info['t_explode']:.2f}s | 起爆位置：({info['explode_pos'][0]:.2f},
        {info['explode_pos'][1]:.2f}, {info['explode_pos'][2]:.2f})m")
    print(f"  投放间隔（与前一枚）：{info['t_drop'] - bombs_info[i-2]['t_drop']:.2f}s" if i>1
        else "  投放间隔：无（第1枚）")
    print("-"*80)

# 6. 按指定格式写入Excel文件（与result.xlsx格式完全一致）
# 创建数据列表，按照Excel模板格式组织
data = [
    {
        "无人机运动方向": round(theta, 2),
        "无人机运动速度 (m/s)": round(v, 2),
        "烟幕干扰弹编号": 1,
        "烟幕干扰弹投放点的x坐标 (m)": round(bombs_info[0]["drop_pos"][0], 2),
        "烟幕干扰弹投放点的y坐标 (m)": round(bombs_info[0]["drop_pos"][1], 2),
        "烟幕干扰弹投放点的z坐标 (m)": round(bombs_info[0]["drop_pos"][2], 2),
        "烟幕干扰弹起爆点的x坐标 (m)": round(bombs_info[0]["explode_pos"][0], 2),
        "烟幕干扰弹起爆点的y坐标 (m)": round(bombs_info[0]["explode_pos"][1], 2),
        "烟幕干扰弹起爆点的z坐标 (m)": round(bombs_info[0]["explode_pos"][2], 2),
        "有效干扰时长 (s)": round(best.value, 2)
    },
    {
        "无人机运动方向": None,

```

```

    "无人机运动速度 (m/s)": None,
    "烟幕干扰弹编号": 2,
    "烟幕干扰弹投放点的x坐标 (m)": round(bombs_info[1]["drop_pos"][0], 2),
    "烟幕干扰弹投放点的y坐标 (m)": round(bombs_info[1]["drop_pos"][1], 2),
    "烟幕干扰弹投放点的z坐标 (m)": round(bombs_info[1]["drop_pos"][2], 2),
    "烟幕干扰弹起爆点的x坐标 (m)": round(bombs_info[1]["explode_pos"][0], 2),
    "烟幕干扰弹起爆点的y坐标 (m)": round(bombs_info[1]["explode_pos"][1], 2),
    "烟幕干扰弹起爆点的z坐标 (m)": round(bombs_info[1]["explode_pos"][2], 2),
    "有效干扰时长 (s)": None
},
{
    "无人机运动方向": None,
    "无人机运动速度 (m/s)": None,
    "烟幕干扰弹编号": 3,
    "烟幕干扰弹投放点的x坐标 (m)": round(bombs_info[2]["drop_pos"][0], 2),
    "烟幕干扰弹投放点的y坐标 (m)": round(bombs_info[2]["drop_pos"][1], 2),
    "烟幕干扰弹投放点的z坐标 (m)": round(bombs_info[2]["drop_pos"][2], 2),
    "烟幕干扰弹起爆点的x坐标 (m)": round(bombs_info[2]["explode_pos"][0], 2),
    "烟幕干扰弹起爆点的y坐标 (m)": round(bombs_info[2]["explode_pos"][1], 2),
    "烟幕干扰弹起爆点的z坐标 (m)": round(bombs_info[2]["explode_pos"][2], 2),
    "有效干扰时长 (s)": None
}
]

# 创建DataFrame并保持列的顺序与模板一致
df = pd.DataFrame(data, columns=[
    "无人机运动方向", "无人机运动速度 (m/s)", "烟幕干扰弹编号",
    "烟幕干扰弹投放点的x坐标 (m)", "烟幕干扰弹投放点的y坐标 (m)", "烟幕干扰弹投放点的z坐标 (m)",
    "烟幕干扰弹起爆点的x坐标 (m)", "烟幕干扰弹起爆点的y坐标 (m)", "烟幕干扰弹起爆点的z坐标 (m)",
    "有效干扰时长 (s)"
])

# 保存到Excel文件
df.to_excel("result.xlsx", index=False)
print(f"\n 结果已按指定格式保存到 'result.xlsx' 文件!")

```

附录 H 问题四–python 源代码

```

import numpy as np
import optuna
import pandas as pd
from openpyxl import Workbook

```

```

# ----- 基础类定义（复用，适配多无人机） -----

class Missile:
    def __init__(self, initial_pos, target_fake):
        self.speed = 300
        self.initial_pos = np.array(initial_pos, dtype=np.float64)
        self.target_fake = np.array(target_fake, dtype=np.float64)
        self.direction = (self.target_fake - self.initial_pos) /
            np.linalg.norm(self.target_fake - self.initial_pos)
        self.total_distance = np.linalg.norm(self.initial_pos - self.target_fake)
        self.time_to_target = self.total_distance / self.speed

    def get_position(self, t):
        if t >= self.time_to_target:
            return self.target_fake.copy()
        return self.initial_pos + self.direction * self.speed * t

class Drone:
    def __init__(self, initial_pos, speed, target_dir):
        """target_dir: 单位方向向量; initial_pos: 每架无人机的初始位置（题目给定）"""
        self.initial_pos = np.array(initial_pos, dtype=np.float64)
        self.speed = speed
        self.direction = np.array(target_dir, dtype=np.float64)
        self.direction /= np.linalg.norm(self.direction) # 确保归一化

    def get_position(self, t):
        return self.initial_pos + self.direction * self.speed * t

    def get_velocity_vector(self):
        return self.direction * self.speed

class SmokeBomb:
    def __init__(self, drop_pos, drop_time, drone_velocity):
        self.drop_pos = np.array(drop_pos, dtype=np.float64)
        self.drop_time = drop_time
        self.drone_velocity = drone_velocity
        self.explode_time = None
        self.explode_pos = None
        self.g = 9.8
        self.sink_speed = 3

    def calculate_explode_position(self, explode_delay):
        self.explode_time = self.drop_time + explode_delay
        time_in_air = explode_delay

# xy方向惯性运动

```

```

xy_displacement = self.drone_velocity * time_in_air
explode_xy = self.drop_pos[:2] + xy_displacement[:2]

# z方向自由落体（落地则无效）
z_fall = 0.5 * self.g * (time_in_air ** 2)
explode_z = self.drop_pos[2] - z_fall
if explode_z < 0:
    raise ValueError("Smoke bomb hits ground before explosion")

self.explode_pos = np.array([explode_xy[0], explode_xy[1], explode_z])
return self.explode_pos

def get_position(self, t):
    if t < self.explode_time:
        return None
    sink_time = t - self.explode_time
    current_z = max(0, self.explode_pos[2] - self.sink_speed * sink_time)
    return np.array([self.explode_pos[0], self.explode_pos[1], current_z])

# ----- 辅助函数（复用） -----
def is_shielding(missile_pos, smoke_pos, target_real, shield_radius=10):
    """判断烟幕是否遮蔽真目标（中心点(0,200,0)）"""
    missile_to_target = target_real - missile_pos
    missile_to_smoke = smoke_pos - missile_pos
    dist_missile_target = np.linalg.norm(missile_to_target)

    if dist_missile_target < 1e-6:
        return False

    projection_ratio = np.dot(missile_to_smoke, missile_to_target) / (dist_missile_target
        ** 2)
    if not (0 < projection_ratio < 1):
        return False

    closest_point = missile_pos + projection_ratio * missile_to_target
    dist_smoke_line = np.linalg.norm(smoke_pos - closest_point)
    return dist_smoke_line <= shield_radius

def merge_intervals(intervals):
    """合并重叠区间，返回并集时长"""
    if not intervals:
        return 0.0
    sorted_intervals = sorted(intervals, key=lambda x: x[0])
    merged = [sorted_intervals[0]]
    for current in sorted_intervals[1:]:

```



```

last = merged[-1]
if current[0] <= last[1]:
merged[-1] = (last[0], max(last[1], current[1]))
else:
merged.append(current)
return sum(end - start for start, end in merged)

# ----- 目标函数（扩展为3架无人机） -----
def objective_3drones_1bomb_each(trial):
"""
优化目标: FY1、FY2、FY3各投1枚弹，最大化遮蔽时间并集
每架无人机参数独立: 速度、方向、投放延迟、起爆延迟
"""
# ----- 1. 定义三架无人机的初始位置（题目给定） -----
drone_initial_positions = {
    "FY1": [17800, 0, 1800],
    "FY2": [12000, 1400, 1400],
    "FY3": [6000, -3000, 700]
}

# ----- 2. 采样三架无人机的参数（独立采样） -----
drones_params = {}

# （1）FY1参数: 方向聚焦175-185°（指向真目标附近）
drones_params["FY1"] = {
    "v": trial.suggest_float("FY1_velocity", 70.0, 140.0),
    "theta": trial.suggest_float("FY1_direction", 175.0, 185.0),
    "t_drop": trial.suggest_float("FY1_t_drop", 0.0, 25.0),
    "t_explode": trial.suggest_float("FY1_t_explode", 0.1, 18.0)
}

# （2）FY2参数: 方向聚焦180-190°（初始在(12000,1400)，指向真目标(0,200)）
drones_params["FY2"] = {
    "v": trial.suggest_float("FY2_velocity", 70.0, 140.0),
    "theta": trial.suggest_float("FY2_direction", 180.0, 190.0),
    "t_drop": trial.suggest_float("FY2_t_drop", 0.0, 30.0),
    "t_explode": trial.suggest_float("FY2_t_explode", 0.1, 15.0) #
    FY2初始z=1400，起爆延迟可稍短
}

# （3）FY3参数: 方向聚焦170-180°（初始在(6000,-3000)，指向真目标(0,200)）
drones_params["FY3"] = {
    "v": trial.suggest_float("FY3_velocity", 70.0, 140.0),
    "theta": trial.suggest_float("FY3_direction", 170.0, 180.0),
    "t_drop": trial.suggest_float("FY3_t_drop", 0.0, 35.0), #
    FY3离目标更近，投放延迟可稍长
}

```

```

        "t_explode": trial.suggest_float("FY3_t_explode", 0.1, 10.0) #
            FY3初始z=700, 起爆延迟更短（避免落地）
    }

    # ----- 3. 初始化仿真对象 -----
    target_fake = [0, 0, 0]
    target_real = [0, 200, 0]
    missile = Missile(initial_pos=[20000, 0, 2000], target_fake=target_fake) # 仅干扰M1
    drones = {}
    bombs = {}

    # ----- 4. 计算每架无人机的烟幕弹状态 -----
    try:
        for drone_name, params in drones_params.items():
            # 解析飞行方向
            theta = params["theta"]
            dir_x = np.cos(np.radians(theta))
            dir_y = np.sin(np.radians(theta))
            drone_dir = [dir_x, dir_y, 0.0]

            # 初始化无人机
            drone = Drone(
                initial_pos=drone_initial_positions[drone_name],
                speed=params["v"],
                target_dir=drone_dir
            )
            drones[drone_name] = drone

            # 计算投放/起爆状态
            t_drop = params["t_drop"]
            t_explode = params["t_explode"]
            drop_pos = drone.get_position(t_drop)
            drone_vel = drone.get_velocity_vector()

            # 初始化烟幕弹
            bomb = SmokeBomb(drop_pos, t_drop, drone_vel)
            bomb.calculate_explode_position(t_explode)
            bombs[drone_name] = bomb
        except Exception as e:
            return 0.0 # 任意无人机/烟幕弹无效, 返回0

    # ----- 5. 仿真三枚烟幕弹的遮蔽区间 -----
    # 仿真结束时间: 导弹到达假目标 或 最后一枚烟幕弹失效
    max_simulation_t = min(
        missile.time_to_target,
        max(bomb.explode_time + 20 for bomb in bombs.values())
    )

```

```

current_t = 0.0
time_step = 0.01
all_intervals = [] # 存储所有烟雾弹的遮蔽区间

while current_t <= max_simulation_t:
    missile_pos = missile.get_position(current_t)

    # 遍历三枚烟雾弹，记录遮蔽区间
    for bomb in bombs.values():
        smoke_pos = bomb.get_position(current_t)
        if smoke_pos is None or current_t > bomb.explode_time + 20:
            continue # 未起爆或已失效

    # 判断遮蔽并记录区间
    if is_shielding(missile_pos, smoke_pos, target_real):
        # 若当前无活跃区间，新建
        if not all_intervals or all_intervals[-1][1] != float('inf'):
            all_intervals.append((current_t, float('inf')))
        else:
            # 若当前有活跃区间，结束并记录
            if all_intervals and all_intervals[-1][1] == float('inf'):
                all_intervals[-1] = (all_intervals[-1][0], current_t)

    current_t += time_step

    # 处理仿真结束时仍在遮蔽的区间
    if all_intervals and all_intervals[-1][1] == float('inf'):
        all_intervals[-1] = (all_intervals[-1][0], max_simulation_t)

    # ----- 6. 合并区间，返回并集时长 -----
    total_shield = merge_intervals(all_intervals)
    return total_shield

def export_to_excel(drones_info, total_shield_time, filename="result2.xlsx"):
    """将优化结果导出到Excel，格式与图片一致"""
    # 创建数据列表
    data = []
    for drone_name, info in drones_info.items():
        row = {
            "无人机编号": drone_name,
            "无人机运动方向(°)": f"{info['theta']:.2f}",
            "无人机运动速度(m/s)": f"{info['v']:.2f}",
            "烟雾干扰弹投放点的x坐标(m)": f"{info['drop_pos'][0]:.2f}",
            "烟雾干扰弹投放点的y坐标(m)": f"{info['drop_pos'][1]:.2f}",
            "烟雾干扰弹投放点的z坐标(m)": f"{info['drop_pos'][2]:.2f}",
            "烟雾干扰弹起爆点的x坐标(m)": f"{info['explode_pos'][0]:.2f}",

```

```

        "烟雾干扰弹起爆点的y坐标(m)": f"{info['explode_pos'][1]:.2f}",
        "烟雾干扰弹起爆点的z坐标(m)": f"{info['explode_pos'][2]:.2f}",
        "有效干扰(s)": f"{total_shield_time:.2f}" if drone_name == "FY1" else "" #
            只在第一行显示总有效干扰时间
    }
    data.append(row)

# 创建DataFrame
df = pd.DataFrame(data)

# 调整列的顺序与图片一致
columns_order = [
    "无人机编号", "无人机运动方向(°)", "无人机运动速度(m/s)",
    "烟雾干扰弹投放点的x坐标(m)", "烟雾干扰弹投放点的y坐标(m)", "烟雾干扰弹投放点的z坐标(m)",
    "烟雾干扰弹起爆点的x坐标(m)", "烟雾干扰弹起爆点的y坐标(m)", "烟雾干扰弹起爆点的z坐标(m)",
    "有效干扰(s)"
]
df = df[columns_order]

# 写入Excel
with pd.ExcelWriter(filename, engine='openpyxl') as writer:
    df.to_excel(writer, index=False, sheet_name='最优策略')

# 调整列宽
worksheet = writer.sheets['最优策略']
for column_cells in worksheet.columns:
    length = max(len(str(cell.value)) for cell in column_cells)
    worksheet.column_dimensions[column_cells[0].column_letter].width = length + 2

# ----- 主函数（运行优化并输出结果） -----
if __name__ == "__main__":
    # 1. 配置贝叶斯优化（参数维度增加，迭代次数适当增加）
    study = optuna.create_study(
        direction="maximize",
        sampler=optuna.samplers.TPESampler(seed=40), # 固定种子，结果可复现
        pruner=optuna.pruners.MedianPruner(n_warmup_steps=30) # 热身30次后剪枝
    )

    # 2. 运行优化（3架无人机×4参数=12维，迭代3000次）
    study.optimize(objective_3drones_1bomb_each, n_trials=3000, show_progress_bar=True)

    # 3. 解析最优参数
    best = study.best_trial
    target_real = [0, 200, 0]
    drone_initial_positions = {"FY1": [17800, 0, 1800], "FY2": [12000, 1400, 1400], "FY3":
        [6000, -3000, 700]}

```

```

drones_info = {}

# 计算每架无人机及烟幕弹的详细状态
for drone_name in ["FY1", "FY2", "FY3"]:
    # 提取最优参数
    v = best.params[f"{drone_name}_velocity"]
    theta = best.params[f"{drone_name}_direction"]
    t_drop = best.params[f"{drone_name}_t_drop"]
    t_explode = best.params[f"{drone_name}_t_explode"]

    # 计算飞行方向和位置
    dir_x = np.cos(np.radians(theta))
    dir_y = np.sin(np.radians(theta))
    drone_dir = [dir_x, dir_y, 0.0]
    drone = Drone(drone_initial_positions[drone_name], v, drone_dir)

    # 计算投放/起爆位置
    drop_pos = drone.get_position(t_drop)
    drone_vel = drone.get_velocity_vector()
    bomb = SmokeBomb(drop_pos, t_drop, drone_vel)
    explode_pos = bomb.calculate_explode_position(t_explode)

    # 存储信息
    drones_info[drone_name] = {
        "v": v, "theta": theta, "t_drop": t_drop, "t_explode": t_explode,
        "drop_pos": drop_pos, "explode_pos": explode_pos, "bomb": bomb
    }

# 4. 输出最优策略
print("\n" + "="*100)
print("问题4: FY1、FY2、FY3三架无人机各投1枚弹的最优策略")
print("="*100)
print(f"总遮蔽时间并集: {best.value:.2f}s")
print("-"*100)

for drone_name, info in drones_info.items():
    print(f"{drone_name}参数: ")
    print(f" 飞行速度: {info['v']:.2f}m/s | 飞行方向: {info['theta']:.2f}°")
    print(f" 投放延迟: {info['t_drop']:.2f}s | 投放位置: ({info['drop_pos'][0]:.2f}, {info['drop_pos'][1]:.2f}, {info['drop_pos'][2]:.2f})m")
    print(f" 起爆延迟: {info['t_explode']:.2f}s | 起爆位置: ({info['explode_pos'][0]:.2f}, {info['explode_pos'][1]:.2f}, {info['explode_pos'][2]:.2f})m")
    print()

# 5. 导出结果到Excel
export_to_excel(drones_info, best.value)
print(f"结果已导出到 result2.xlsx")

```

附录 I 问题五—python 源代码

```
import numpy as np
from scipy.optimize import linear_sum_assignment
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import random
import tqdm
from matplotlib.animation import FuncAnimation

# ----- 1. 目标与载体位置定义 -----
class MissileDefenseSystem:
    def __init__(self):
        # 导弹初始位置
        self.missiles = {
            'Missile_1': np.array([20000, 0, 2000]),
            'Missile_2': np.array([19000, 600, 2100]),
            'Missile_3': np.array([18000, -600, 1900])
        }

        # 无人机初始位置
        self.drones = {
            'Drone_FY1': np.array([17800, 0, 1800]),
            'Drone_FY2': np.array([12000, 1400, 1400]),
            'Drone_FY3': np.array([6000, -3000, 700]),
            'Drone_FY4': np.array([11000, 2000, 1800]),
            'Drone_FY5': np.array([13000, -2000, 1300])
        }

        # 目标位置
        self.fake_target = np.array([0, 0, 0])
        self.real_target = np.array([0, 200, 0])

        # 物理参数
        self.gravity = 9.8          # 重力加速度(m/s2)
        self.missile_speed = 300    # 导弹速度(m/s)
        self.smoke_sink_speed = 3   # 烟幕沉降速度(m/s)
        self.smoke_radius = 10      # 烟幕有效半径(m)
        self.smoke_duration = 20    # 烟幕持续时间(s)

        # 分配结果
        self.assignment_result = None
        self.cost_matrix = None
```

```

# ----- 2. 几何计算工具函数 -----
def point_to_line_distance(self, point, line_point, line_dir):
    """计算点到直线的垂直距离"""
    vec_to_point = point - line_point
    cross_product = np.linalg.norm(np.cross(vec_to_point, line_dir))
    return cross_product / np.linalg.norm(line_dir)

# ----- 3. 导弹-无人机分配算法 -----
def assign_drones_to_missiles(self):
    """使用匈牙利算法进行无人机-导弹分配"""
    missile_names = list(self.missiles.keys())
    drone_names = list(self.drones.keys())

    # 初始化成本矩阵
    self.cost_matrix = np.zeros((len(missile_names), len(drone_names)))

    # 填充成本矩阵：无人机到导弹飞行直线的距离
    for i, missile_id in enumerate(missile_names):
        missile_pos = self.missiles[missile_id]
        missile_dir = self.fake_target - missile_pos # 导弹飞行方向

        for j, drone_id in enumerate(drone_names):
            drone_pos = self.drones[drone_id]
            self.cost_matrix[i, j] = self.point_to_line_distance(
                drone_pos, missile_pos, missile_dir)

    # 应用匈牙利算法
    row_ind, col_ind = linear_sum_assignment(self.cost_matrix)

    # 整理分配结果
    self.assignment_result = []
    for r, c in zip(row_ind, col_ind):
        self.assignment_result.append({
            'missile': missile_names[r],
            'drone': drone_names[c],
            'distance': round(self.cost_matrix[r, c], 2)
        })

    return self.assignment_result

# ----- 4. 烟幕遮蔽效果计算 -----
def calculate_smoke_effect(self, drone_dir, drone_speed, drop_time, blast_delay,
                           visualize=False, verbose=False):
    """计算烟幕遮蔽效果"""
    # 无人机参数
    drone_dir_norm = np.array(drone_dir) / np.linalg.norm(drone_dir)

```

```

drone_vel = drone_speed * drone_dir_norm

# 无人机初始位置 (以FY1为例)
drone_init_pos = self.drones['Drone_FY1']

# 计算投放点和起爆点
drop_pos = drone_init_pos + drone_vel * drop_time
vertical_drop = 0.5 * self.gravity * blast_delay**2
blast_pos = np.array([
    drop_pos[0] + drone_vel[0] * blast_delay,
    drop_pos[1] + drone_vel[1] * blast_delay,
    drop_pos[2] - vertical_drop
])

# 导弹参数 (以Missile_1为例)
missile_init_pos = self.missiles['Missile_1']
missile_dir = (self.fake_target - missile_init_pos) / np.linalg.norm(
    self.fake_target - missile_init_pos)
missile_vel = self.missile_speed * missile_dir

# 导弹到达目标时间
time_to_target = (self.fake_target[0] - missile_init_pos[0]) / missile_vel[0]

# 时间参数
blast_time = drop_time + blast_delay
max_smoke_time = min(self.smoke_duration, time_to_target - blast_time)

if max_smoke_time <= 0:
    return 0.0, [], [], blast_pos

# 计算有效遮蔽时间
time_step = 0.1
total_effective = 0.0
missile_positions = []
smoke_positions = []

for t in np.arange(0, max_smoke_time, time_step):
    # 当前时间点
    current_time = blast_time + t
    if current_time > time_to_target:
        break

    # 导弹位置
    missile_pos = missile_init_pos + missile_vel * current_time
    missile_positions.append(missile_pos)

    # 烟幕位置

```



```

smoke_pos = np.array([
    blast_pos[0],
    blast_pos[1],
    blast_pos[2] - self.smoke_sink_speed * t
])
smoke_positions.append(smoke_pos)

# 判断遮蔽效果
target_dir = (self.real_target - missile_pos) / np.linalg.norm(
    self.real_target - missile_pos)
dist_to_line = self.point_to_line_distance(smoke_pos, missile_pos, target_dir)

# 烟幕是否在导弹和目标之间
vec_missile_smoke = smoke_pos - missile_pos
vec_missile_target = self.real_target - missile_pos
dot_product = np.dot(vec_missile_smoke, vec_missile_target)
is_between = dot_product > 0

# 导弹是否在烟幕内
in_smoke = np.linalg.norm(missile_pos - smoke_pos) <= self.smoke_radius

if (dist_to_line <= self.smoke_radius and is_between) or in_smoke:
    total_effective += time_step

# 可视化
if visualize:
    self._visualize_simulation(missile_positions, smoke_positions, blast_pos)

return total_effective, missile_positions, smoke_positions, blast_pos

def _visualize_simulation(self, missile_traj, smoke_traj, blast_pos):
    """可视化模拟结果"""
    fig = plt.figure(figsize=(12, 10))
    ax = fig.add_subplot(111, projection='3d')

    # 绘制轨迹
    if missile_traj:
        missile_arr = np.array(missile_traj)
        ax.plot(missile_arr[:, 0], missile_arr[:, 1], missile_arr[:, 2],
            'r-', label='导弹轨迹')

    if smoke_traj:
        smoke_arr = np.array(smoke_traj)
        ax.plot(smoke_arr[:, 0], smoke_arr[:, 1], smoke_arr[:, 2],
            'b-', label='烟幕轨迹')

    # 标记关键点

```

```

ax.scatter(*self.fake_target, color='red', s=100, label='假目标', marker='x')
ax.scatter(*self.real_target, color='blue', s=100, label='真目标', marker='o')
ax.scatter(*self.missiles['Missile_1'], color='orange', s=80, label='导弹起点')
ax.scatter(*blast_pos, color='green', s=80, label='起爆点')

# 设置图表
ax.set_xlabel('X (m)')
ax.set_ylabel('Y (m)')
ax.set_zlabel('Z (m)')
ax.set_title('烟雾遮蔽效果模拟')
ax.legend()
ax.grid(True)
plt.show()

# ----- 5. 遗传算法优化 -----
def optimize_parameters(self, pop_size=100, generations=50):
    """使用遗传算法优化烟雾投放参数"""
    # 参数范围：方向角(0-360)，速度(80-120)，投放时间(0-30)，起爆延迟(0-15)
    param_ranges = [(0, 360), (80, 120), (0, 30), (0, 15)]

    # 创建初始种群
    population = self._create_population(pop_size, param_ranges)

    best_fitness = 0
    best_params = None

    print("开始遗传算法优化...")
    for gen in tqdm.tqdm(range(generations)):
        # 评估适应度
        fitnesses = []
        for individual in population:
            dir_angle = individual[0]
            # 将角度转换为方向向量
            dir_vec = [np.cos(np.radians(dir_angle)),
                       np.sin(np.radians(dir_angle)), 0]
            fitness = self.calculate_smoke_effect(
                dir_vec, individual[1], individual[2], individual[3])[0]
            fitnesses.append(fitness)

        # 记录最佳解
        current_best_idx = np.argmax(fitnesses)
        if fitnesses[current_best_idx] > best_fitness:
            best_fitness = fitnesses[current_best_idx]
            best_params = population[current_best_idx]

    # 选择
    selected = self._selection(population, fitnesses)

```

```

# 交叉和变异
population = self._crossover_and_mutate(selected, param_ranges)

return best_params, best_fitness

def _create_population(self, size, ranges):
    """创建初始种群"""
    population = []
    for _ in range(size):
        individual = [
            random.uniform(ranges[0][0], ranges[0][1]), # 方向角
            random.uniform(ranges[1][0], ranges[1][1]), # 速度
            random.uniform(ranges[2][0], ranges[2][1]), # 投放时间
            random.uniform(ranges[3][0], ranges[3][1]) # 起爆延迟
        ]
        population.append(individual)
    return population

def _selection(self, population, fitnesses):
    """选择操作"""
    selected = []
    # 轮盘赌选择
    total_fitness = sum(fitnesses) if sum(fitnesses) > 0 else 1
    probabilities = [f / total_fitness for f in fitnesses]

    for _ in range(len(population)):
        selected_idx = np.random.choice(len(population), p=probabilities)
        selected.append(population[selected_idx])
    return selected

def _crossover_and_mutate(self, population, ranges, crossover_rate=0.7,
                          mutation_rate=0.2):
    """交叉和变异操作"""
    new_population = []
    for i in range(0, len(population), 2):
        parent1 = population[i]
        parent2 = population[i+1] if i+1 < len(population) else population[0]

        # 交叉
        if random.random() < crossover_rate:
            point = random.randint(1, 3)
            child1 = parent1[:point] + parent2[point:]
            child2 = parent2[:point] + parent1[point:]
        else:
            child1, child2 = parent1.copy(), parent2.copy()

```

```

# 变异
child1 = self._mutate(child1, ranges, mutation_rate)
child2 = self._mutate(child2, ranges, mutation_rate)

new_population.extend([child1, child2])

return new_population[:len(population)]

def _mutate(self, individual, ranges, mutation_rate):
    """变异操作"""
    mutated = individual.copy()
    for i in range(len(mutated)):
        if random.random() < mutation_rate:
            mutated[i] = random.uniform(ranges[i][0], ranges[i][1])
    return mutated

# ----- 6. 动画展示 -----
def animate_simulation(self, params):
    """创建动画展示"""
    dir_angle, speed, drop_time, blast_delay = params
    dir_vec = [np.cos(np.radians(dir_angle)), np.sin(np.radians(dir_angle)), 0]

    # 获取模拟数据
    _, missile_traj, smoke_traj, blast_pos = self.calculate_smoke_effect(
        dir_vec, speed, drop_time, blast_delay, visualize=False)

    if not missile_traj or not smoke_traj:
        print("没有足够的模拟数据用于动画展示")
        return

    # 创建动画
    fig = plt.figure(figsize=(12, 10))
    ax = fig.add_subplot(111, projection='3d')

    # 初始化轨迹线
    missile_line, = ax.plot([], [], [], 'r-', label='导弹轨迹')
    smoke_line, = ax.plot([], [], [], 'b-', label='烟幕轨迹')

    # 初始化点
    missile_point, = ax.plot([], [], [], 'ro', markersize=8)
    smoke_point, = ax.plot([], [], [], 'bo', markersize=8)

    # 固定点
    ax.scatter(*self.fake_target, color='red', s=100, label='假目标', marker='x')
    ax.scatter(*self.real_target, color='blue', s=100, label='真目标', marker='o')
    ax.scatter(*self.missiles['Missile_1'], color='orange', s=80, label='导弹起点')
    ax.scatter(*blast_pos, color='green', s=80, label='起爆点')

```

```

# 设置坐标轴范围
all_points = np.concatenate([missile_traj, smoke_traj])
ax.set_xlim(all_points[:,0].min()*0.9, all_points[:,0].max()*1.1)
ax.set_ylim(all_points[:,1].min()*0.9, all_points[:,1].max()*1.1)
ax.set_zlim(all_points[:,2].min()*0.9, all_points[:,2].max()*1.1)

ax.set_xlabel('X (m)')
ax.set_ylabel('Y (m)')
ax.set_zlabel('Z (m)')
ax.set_title('烟雾遮蔽动态模拟')
ax.legend()

# 动画更新函数
def update(frame):
    missile_line.set_data(missile_traj[:frame,0], missile_traj[:frame,1])
    missile_line.set_3d_properties(missile_traj[:frame,2])
    missile_point.set_data([missile_traj[frame,0]], [missile_traj[frame,1]])
    missile_point.set_3d_properties([missile_traj[frame,2]])

    smoke_line.set_data(smoke_traj[:frame,0], smoke_traj[:frame,1])
    smoke_line.set_3d_properties(smoke_traj[:frame,2])
    smoke_point.set_data([smoke_traj[frame,0]], [smoke_traj[frame,1]])
    smoke_point.set_3d_properties([smoke_traj[frame,2]])

    return missile_line, smoke_line, missile_point, smoke_point

# 创建动画
anim = FuncAnimation(
    fig, update, frames=min(len(missile_traj), len(smoke_traj)),
    interval=50, blit=True
)

plt.show()
return anim

# ----- 主函数 -----
def main():
    # 创建系统实例
    system = MissileDefenseSystem()

    # 执行无人机-导弹分配
    print("=== 无人机-导弹分配结果 ===")
    assignments = system.assign_drones_to_missiles()
    for i, assignment in enumerate(assignments):
        print(f"{i+1}. {assignment['missile']} 分配给 {assignment['drone']}, "
              f"垂直距离: {assignment['distance']} m")

```

```

# 优化烟幕投放参数
best_params, best_effect = system.optimize_parameters(pop_size=50, generations=30)
print("\n=== 优化结果 ===")
print(f"最佳参数: 方向角={best_params[0]:.1f}°, 速度={best_params[1]:.1f}m/s, "
      f"投放时间={best_params[2]:.1f}s, 起爆延迟={best_params[3]:.1f}s")
print(f"最大有效遮蔽时间: {best_effect:.2f}s")

# 动画展示最佳方案
if best_params:
    print("\n正在展示动态模拟...")
    dir_vec = [np.cos(np.radians(best_params[0])),
               np.sin(np.radians(best_params[0])), 0]
    system.calculate_smoke_effect(dir_vec, best_params[1], best_params[2],
                                  best_params[3], visualize=True)
    system.animate_simulation(best_params)

if __name__ == "__main__":
    main()

```

附录 J python 环境配置的说明

```

#关于代码包中python脚本的运行说明:
#代码脚本包括q1.py q2.py q3.py q4.py
#python版本:
#3.13.5.final.0
#使用conda虚拟环境,所必要的包有:numpy,matplotlib,torch,optuna,
#openpyxl,pandas

```